

Fundamentos de Teoría de la Computación

Resumen teorías - Parte 1 - Computabilidad y Complejidad

Clase 1 - La Máquina de Turing	6
Conceptos matemáticos básicos	6
Conjunto	6
Función	6
Alfabeto	6
Lenguaje	7
Grafo	7
Fórmula booleana	8
Máquina de Turing	8
Ejemplos	8
Notas	10
Problemas no decidibles	11
Problemas no computables	11
Tesis de Church-Turing	12
Funcionamiento	12
Formalmente	13
MT con varias cintas	14
Características	14
Simulación de una MT con varias cintas mediante una MT con 1 cinta	15
MT no determinística (MTN)	16
Modelos equivalentes de MT	16
Anexo	17
Acerca de los lenguajes	17
Otras visiones de MT	18
Calculadora	18
Generadora	18
Clase 2 - La jerarquía de la computabilidad	20
Problemas decidibles	20
Problemas computables no decidibles	20
Problemas no computables	21
Jerarquía de la computabilidad	21
Definiciones	22
Σ	22
Σ^*	22
$\mathcal{Q}$	22
Lenguaje recursivo	22

Lenguaje recursivamente enumerable	22
Clase R	23
Propiedad 1	23
Prueba	23
Construcción de la MT M2	23
Conclusión	24
Propiedad 2	24
Prueba	24
Construcción de la MT M	25
Clase RE	26
Propiedad 3	26
Prueba	26
Construcción de la MT M	27
Segunda versión de la jerarquía de la computabilidad	28
Propiedad 4	29
Prueba	30
Conclusión	30
Las cuatro regiones de la jerarquía de la computabilidad	31
Región 1 (lenguajes con MT que siempre paran)	31
Región 2 (lenguajes con MT)	31
Región 3 (lenguajes sin MT, con complementos con MT)	31
Región 4 (lenguajes sin MT, con complementos sin MT)	31
Clase 3 - Indecibilidad	32
Máquina de Turing universal	32
Codificación de una máquina de Turing y una cadena de entrada	32
Enumeración de las máquinas de Turing	33
Prueba de que $R \subset RE \subset \mathcal{Q}$	34
Probamos que D está en RE	35
Probamos que DC no está en RE	36
Anexo	37
$ \mathcal{Q} = R > RE = N $	37
Halting Problem	38
Clase 4 - Las reducciones y otros temas de computabilidad	39
Reducción de lenguajes	39
Caso 1: $L2 \in R$	39
Caso 2: $L2 \in RE$	40
Propiedades de las reducciones	41
Reflexividad	41
Transitividad	42
Otra propiedad	42
No se cumple la simetría	42
Otra manera de probar que HP está entre los lenguajes más difíciles de RE	42
Idea general de la prueba	43
Anexo	44

Máquinas de Turing restringidas	44
Autómatas finitos	44
Características de los AF	44
Autómatas con pila	45
Características de los AP	46
Jerarquía de los lenguajes de Chomsky	47
Enumeración de los lenguajes recursivamente enumerables	47
Todo lenguaje L de RE se puede enumerar	47
Todo lenguaje L de R se puede enumerar en el orden canónico	48
Clase 5 - Tiempo polinomial y no polinomial	49
Complejidad temporal	49
Clase P	52
Ejemplo de lenguaje en P	52
Primera versión de la jerarquía temporal	53
Tesis Fuerte de Church-Turing (IMPORTANTE)	53
Ejemplo de lenguaje que no está en P	53
Sin embargo...	54
Segunda versión de la jerarquía temporal (acotada a la clase EXP)	55
P vs NP	55
Clase 6 - NP-completitud	57
Jerarquía temporal hasta ahora	57
Lenguajes NP-completos	57
Definición de NP-completitud	58
¿Cómo encontrar un lenguaje L que sea NP-completo?	59
Observaciones	60
La clase NPI	61
Última versión de la jerarquía temporal	62
Clase 7 - La jerarquía espacio temporal y otros temas de complejidad computacional	63
Complejidad espacial	63
La clase PSPACE	65
Ejemplo de lenguaje en la clase PSPACE	65
La clase LOGSPACE	66
La jerarquía espacial	67
La jerarquía espacio-temporal	68
Complejidad temporal de búsqueda	69
Aproximaciones polinomiales	69
Lenguajes con algoritmos paralelos eficientes	70
MT probabilísticas o MTP (algoritmos aleatorios)	71
Máquinas cuánticas o MC (algoritmos cuánticos)	71
Ejemplo de puertas cuánticas y computación cuántica	73
Las distintas jerarquías	74

Clase 1 - La Máquina de Turing

Conceptos matemáticos básicos

Conjunto

Un conjunto es una colección de elementos

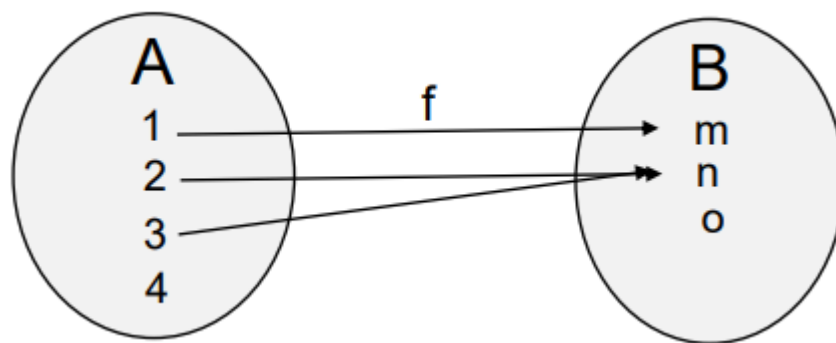
Por ejemplo, $A = \{a, b, c\}$ es un conjunto de tres elementos: a , b y c . Se dice que a pertenece a A ($a \in A$)

Dos conjuntos A y B son iguales ($A = B$) si tienen los mismos elementos

Un conjunto A es un subconjunto de un conjunto B o está incluido en B ($A \subseteq B$) si los elementos de A pertenecen a B , y está incluido estrictamente en B ($A \subset B$) si $A \subseteq B$ y $A \neq B$. Por ejemplo, $\{1, 2\} \subseteq \{1, 2, 3, 5\}$. Y también se cumple $\{1, 2\} \subset \{1, 2, 3, 5\}$

Función

Una función de un conjunto A a un conjunto B ($f : A \rightarrow B$) es un conjunto de pares (a, b) , con $a \in A$ y $b \in B$, tales que a lo sumo existe un par (a, b) para todo $a \in A$



El conjunto A es el dominio y el conjunto B es el codominio de la función f

Alfabeto

Un alfabeto es un conjunto finito de símbolos

Por ejemplo, $\Sigma = \{a, b, c\}$ es un alfabeto de tres símbolos

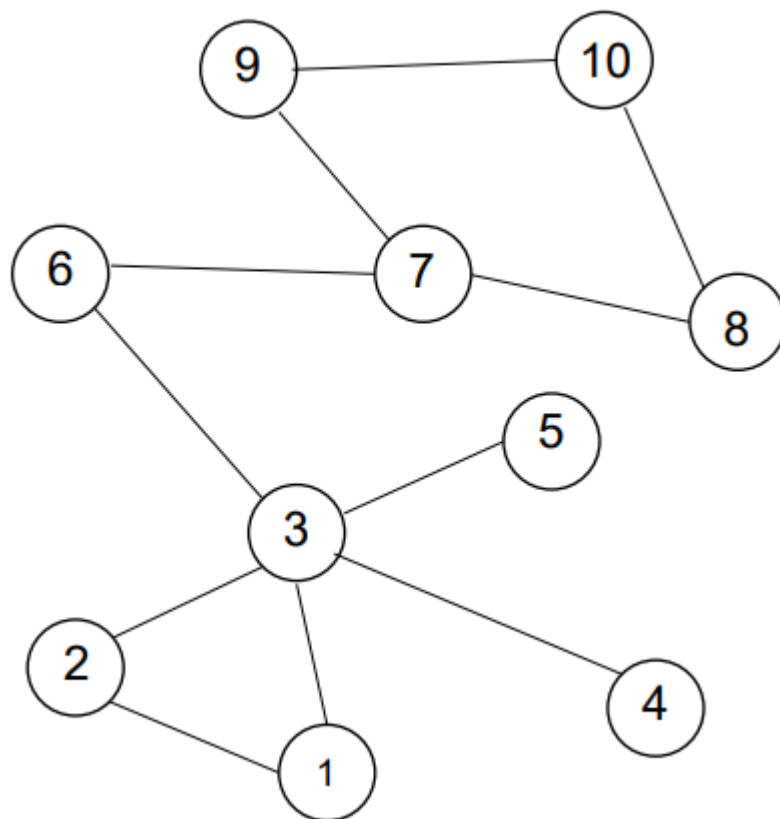
Lenguaje

Un lenguaje con alfabeto Σ es un conjunto de cadenas finitas de símbolos de Σ

Por ejemplo, si $\Sigma = \{a, b, c\}$, un lenguaje L con alfabeto Σ es $L = \{aaa, b, ababab, ccb\}$

Grafo

Un grafo es un par (V, E) , tal que V es un conjunto de vértices y E es un conjunto de arcos



El grafo tiene 10 vértices y 11 arcos

Vértices como 1 y 2 son adyacentes

Arcos como $(1, 2)$ y $(2, 3)$ son adyacentes

La secuencia de vértices $(1, 2, 3, 6, 7, 9, 10)$ representa un camino del vértice 1 al vértice 10

Fórmula booleana

Una fórmula booleana es una fórmula lógica que se obtiene combinando variables booleanas, de valor de verdad verdadero (V) o falso (F), con los operadores lógicos and ($\wedge$), or ($\vee$) y not ($\neg$)

Una fórmula booleana ϕ es satisfactible si existe al menos una asignación de valores de verdad que la evalúa verdadera

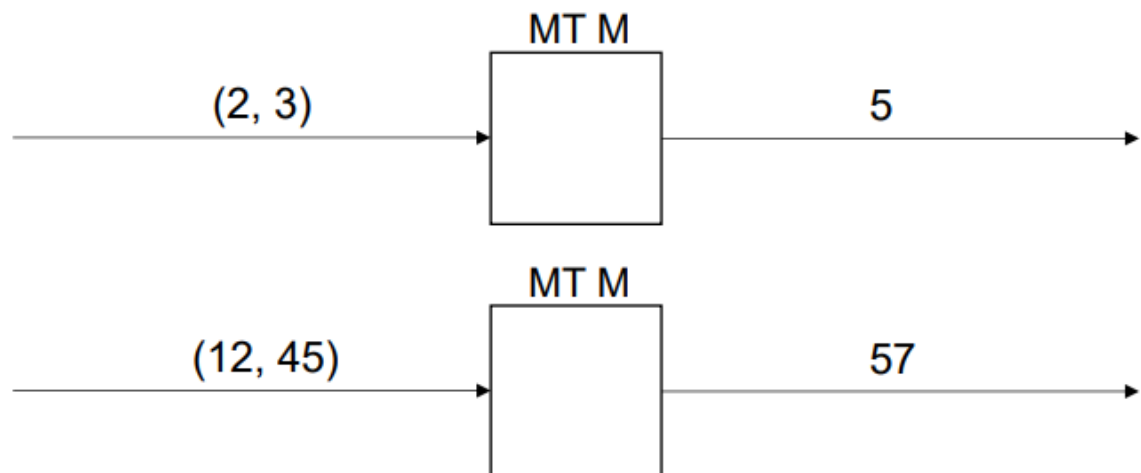
Por ejemplo, la fórmula $\phi = (x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2)$ es satisfactible, porque para $x_1=V$ y $x_2=F$ es verdadera

Máquina de Turing

Modelización muy simple de una computadora, aceptada universalmente para estudiar la computabilidad y la complejidad computacional de los problemas

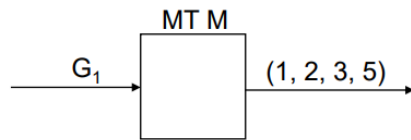
Ejemplos

- MT M que resuelve el problema de la suma de dos números naturales

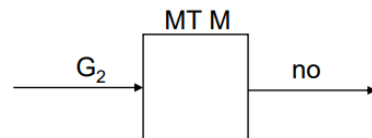
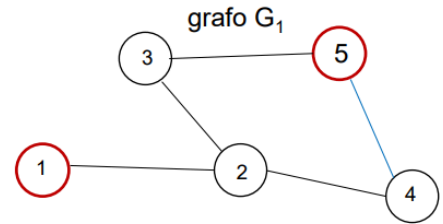


M se comporta de esta manera a partir de todo par de números naturales

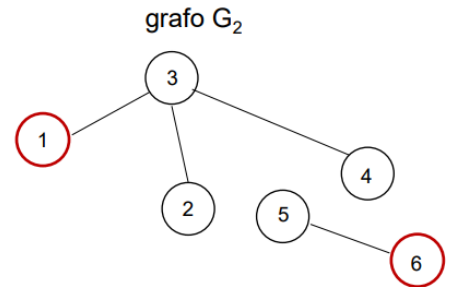
- MT M que resuelve el problema de encontrar en un grafo no dirigido un camino del vértice inicial al vértice final (problema de búsqueda, como el del ejemplo anterior)



Cuando hay solución, la MT M devuelve **alguna**.

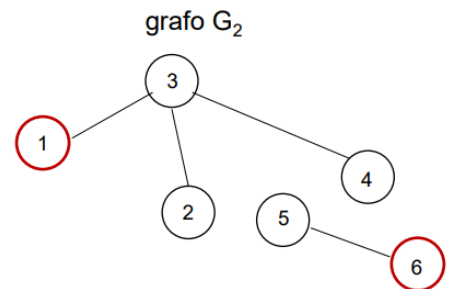
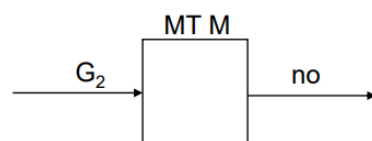
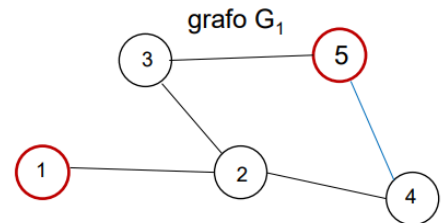
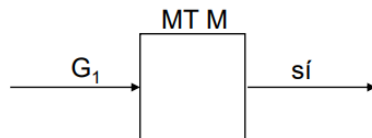


Cuando no hay solución, la MT M **responde no**.



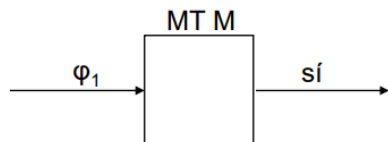
En general consideraremos grafos no dirigidos.

- Problema similar al anterior, pero más simple: MT M que decide si un grafo tiene un camino del vértice inicial al vértice final (problema de decisión)



En este tipo de problemas, la MT sólo responde **sí o no** (**acepta o rechaza** su entrada).

- MT M que decide si una fórmula booleana es satisfactible (otro ejemplo de problema de decisión):

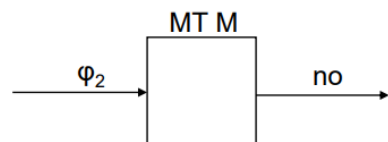


$$\varphi_1 = (x_1 \vee x_2) \wedge (x_3 \wedge x_4)$$

Por ejemplo, la asignación $\mathcal{A} = (V, F, V, V)$ satisface φ_1 :

$$\varphi_1 = (x_1 \vee x_2) \wedge (x_3 \wedge x_4)$$

$\vee \quad V \quad F \quad V \quad \vee \quad V \quad V \quad V$



$$\varphi_2 = x_1 \wedge \neg x_1$$

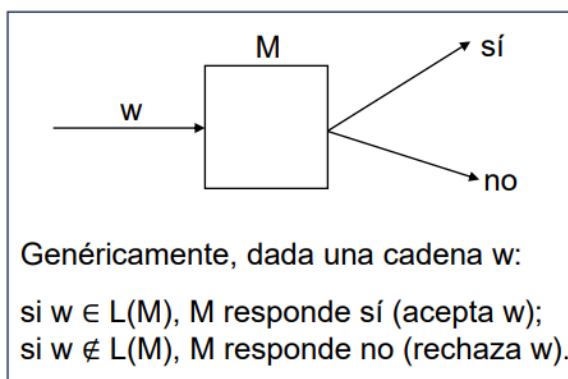
No existe asignación que satisfaga φ_2 .

$$\varphi_2 = x_1 \wedge \neg x_1$$

$\vee \quad F \quad F \quad \vee$
 $F \quad F \quad V \quad F$

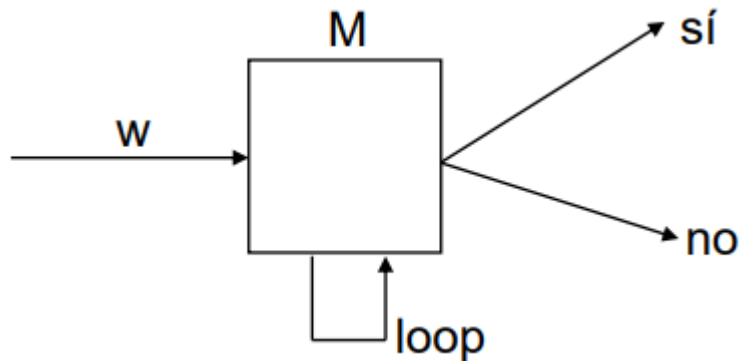
Notas

- Vamos a considerar en general problemas de decisión, no problemas de búsqueda. Por lo tanto, las máquinas de Turing sólo van a responder sí o no (van a aceptar o rechazar). Así se simplifican las descripciones pero sin perder generalidad
- Con esta visión, una MT M acepta o reconoce un lenguaje: el lenguaje formado por las cadenas que M acepta (el lenguaje de M son todas las cadenas que acepta, es decir, todas para las que responde que sí)
- Vamos a denotar con $L(M)$ al lenguaje que acepta la MT M
- Por ejemplo:
 - En el problema de los grafos, $L(M) = \{G_1, G_2, G_3, \dots\}$, donde G_i es un grafo con un camino del primero al último vértice
 - En el problema de las fórmulas booleanas, $L(M) = \{\varphi_1, \varphi_2, \varphi_3, \dots\}$, donde φ_i es una fórmula booleana satisfactible



Problemas no decidibles

El caso más general de MT corresponde al siguiente esquema:

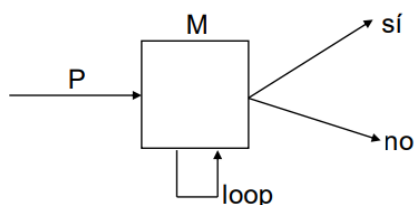


Es decir: si w pertenece a $L(M)$, M responde sí, y si w no pertenece a $L(M)$, en el mejor de los casos M responde no, pero en el peor de los casos M no responde nada porque no para, “loopea”

Lamentablemente, existen problemas que sólo cuentan con MT de este tipo

Son problemas computables no decidibles (los que vimos antes son problemas computables decidibles)

Ejemplo clásico: el problema de decidir si un programa P para (halting problem)



La mejor MT M posible:

Si P para, responde sí.

Si P no para, responde no o loopea.

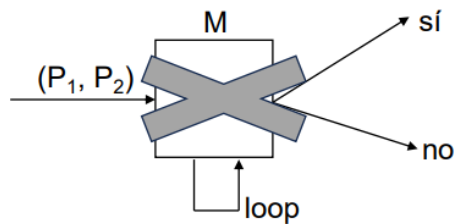
Es decir, en este caso **no existe una MT M que pare siempre.**

Problemas no computables

Existen problemas que directamente no tienen MT que los resuelvan

Es decir: dichos problemas ni siquiera cuentan con MT M que respondan sí en todos los casos positivos (que es lo mínimo que se puede esperar de una MT)

Ejemplo clásico: el problema de decidir la equivalencia de dos programas P_1 y P_2

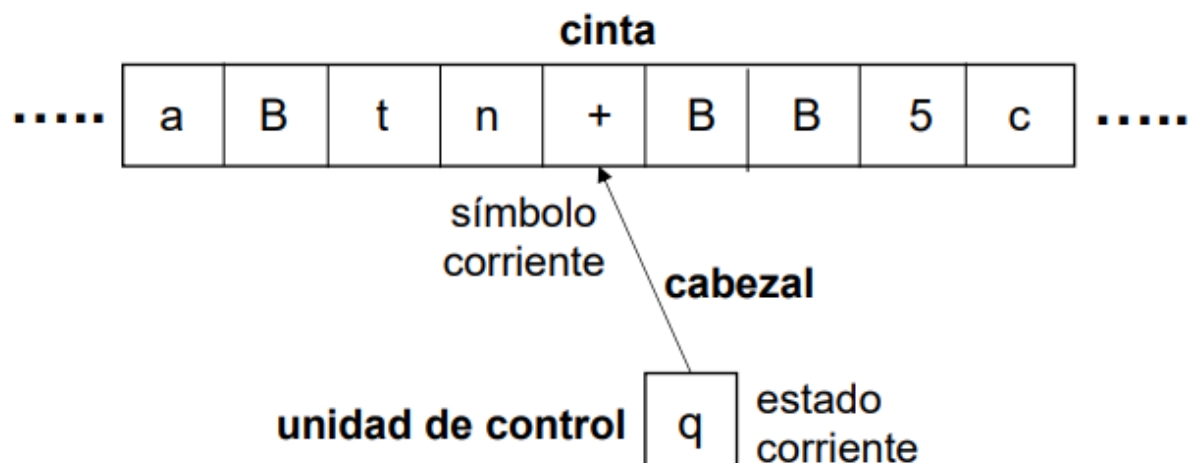


No existe ni siquiera una MT que responda sí para todos los casos en que P_1 sea equivalente a P_2 (lo mínimo que se puede pedir), y por eso **este problema no es computable**.

Tesis de Church-Turing

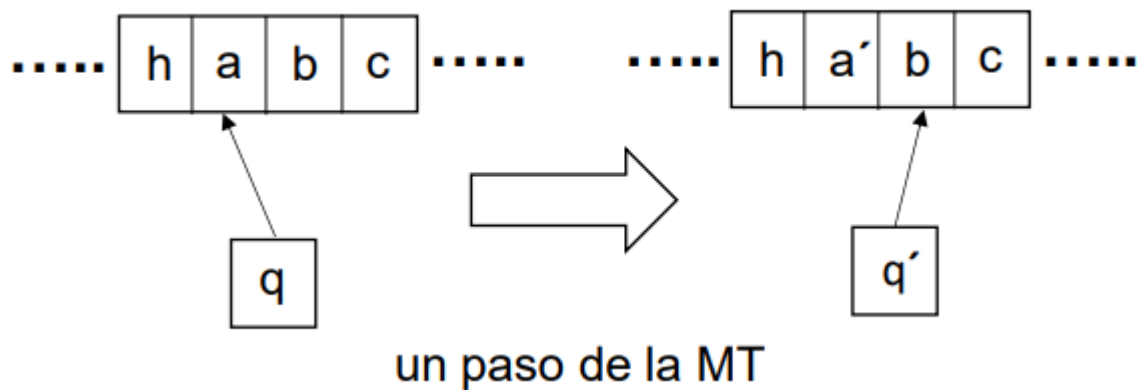
Todo dispositivo computacional físicamente realizable puede ser simulado por una MT

Funcionamiento

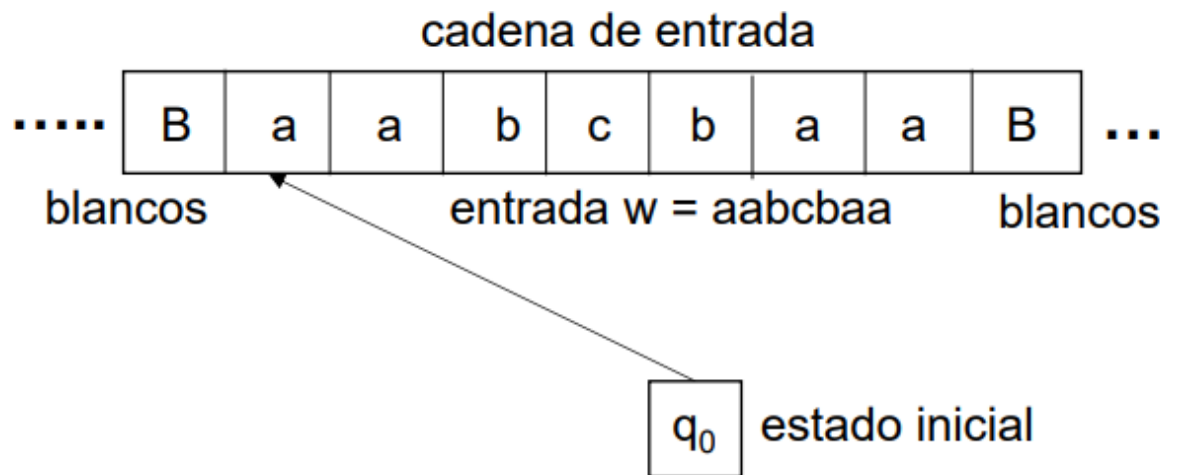


La cinta almacena símbolos de un alfabeto Γ , y la unidad de control tiene siempre un estado de un conjunto de estados Q

En un paso, una MT puede: modificar el símbolo corriente, modificar el estado corriente, y moverse un lugar a la derecha o a la izquierda (o quedarse en su sitio)



Al inicio, la entrada se delimita por blancos, y el cabezal apunta al primer símbolo de la izquierda



Formalmente

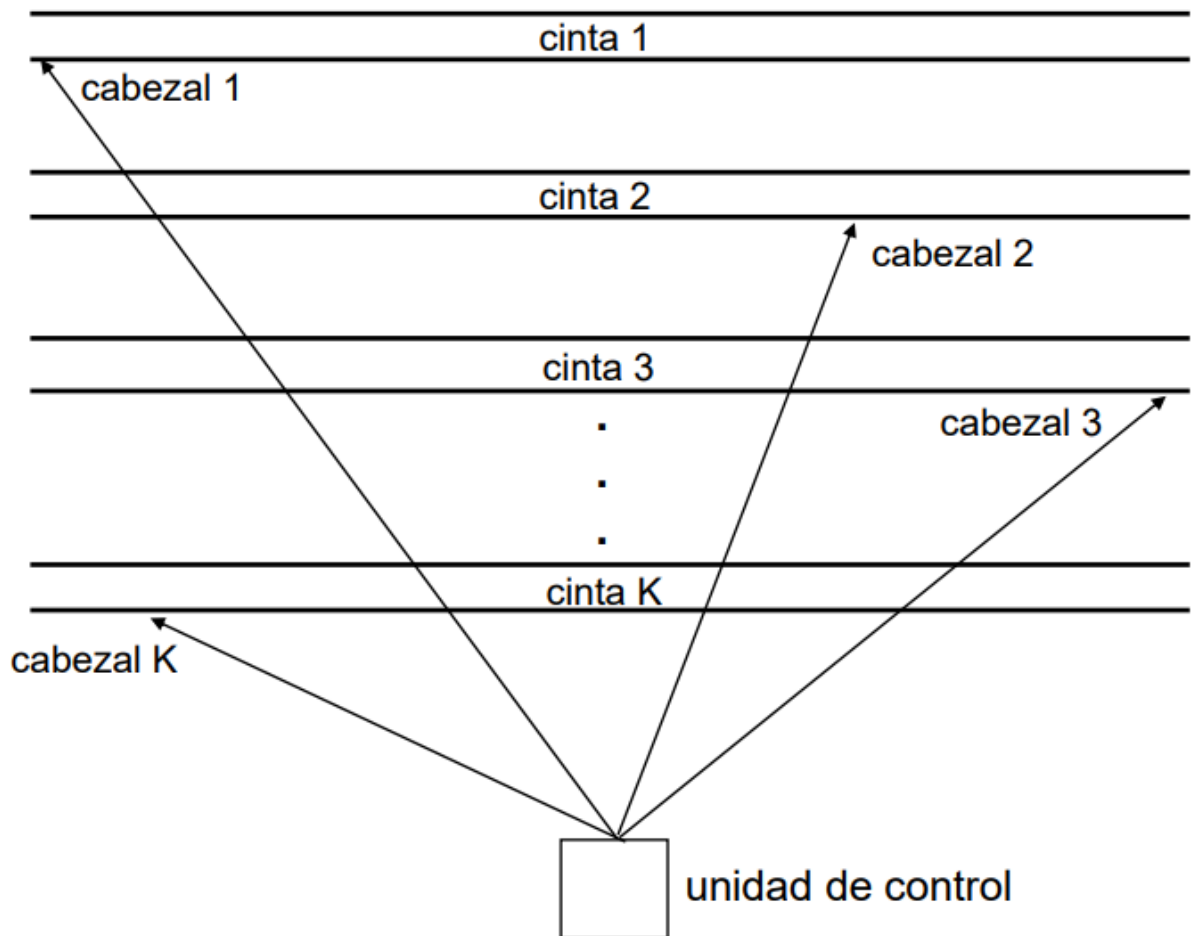
Una MT M es una tupla $(Q, \Gamma, \delta, q_0, q_A, q_R)$, donde:

- Q es el conjunto de estados de M
- Γ es el alfabeto (conjunto de símbolos) de M . Γ incluye al símbolo blanco B .
Las cadenas de entrada no admiten blancos
- q_0 es el estado inicial de M
- q_A y q_R son los estados finales de M (de aceptación y de rechazo)
- δ es la función de transición de M (especifica su comportamiento): Dado un estado corriente de Q , y un símbolo corriente de Γ , la MT M :
 - Pasa eventualmente a un nuevo estado de Q
 - Modifica eventualmente el símbolo corriente de Γ

- Y se mueve un lugar a la derecha (R), a la izquierda (L), o no se mueve (S)

La máquina para si en algún momento alcanza el estado q_A o q_R

MT con varias cintas



Características

La primera cinta contiene la entrada

La MT en un solo paso puede modificar el estado corriente, los símbolos corrientes de todas las cintas, y moverse distinto en cada cinta

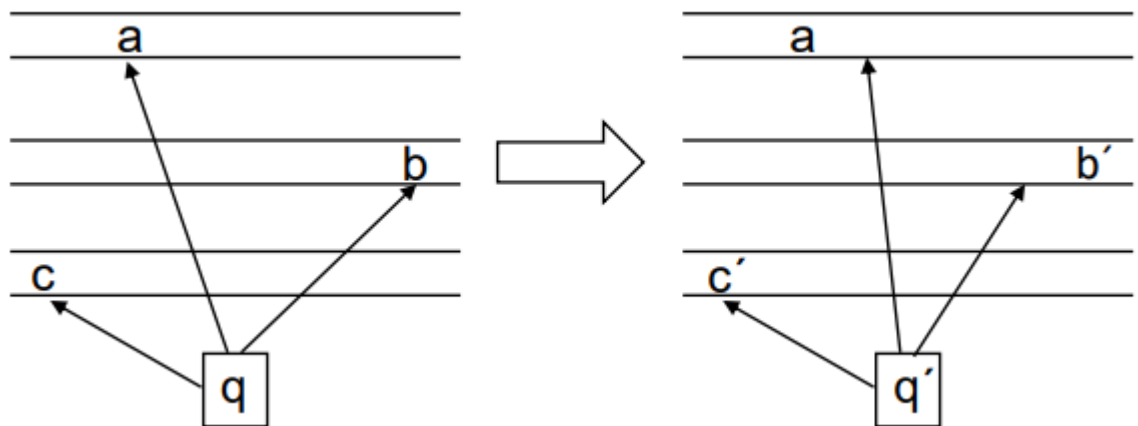
Más en detalle, en un paso:

1. Lee un estado y una tupla de símbolos (los apuntados por los cabezales)
2. Modifica eventualmente el estado corriente

3. Modifica eventualmente los símbolos corrientes de las cintas
4. Se mueve independientemente en cada cinta (a la derecha, a la izquierda, o no se mueve)

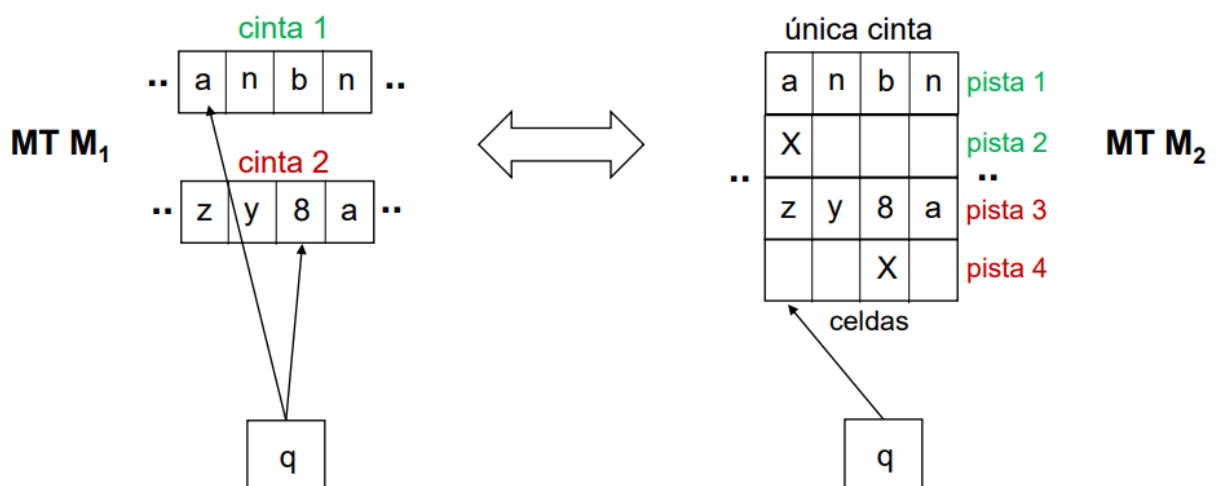
Por ejemplo, para el caso de 3 cintas, una transición podría ser:

$$\delta(q, (a, b, c)) = (q', (a, R), (b', L), (c', S))$$



Simulación de una MT con varias cintas mediante una MT con 1 cinta

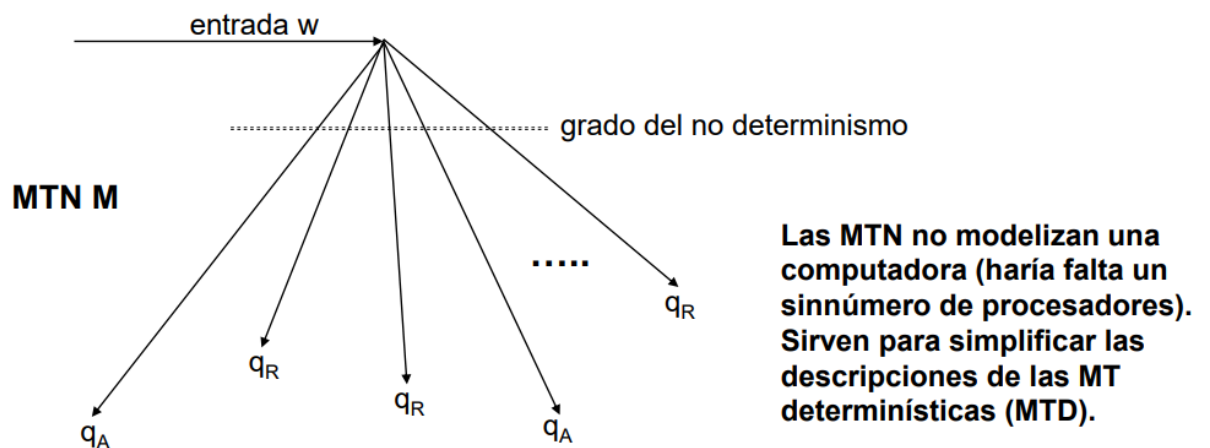
Aunque no es intuitivo, una MT con varias cintas no tiene más potencia computacional que una MT con 1 cinta (lo que hace una MT con K cintas lo puede hacer una MT con 1 cinta). Si va a ser más veloz, pero no más potente...



Idea general: uso de cintas con pistas: en el ejemplo, las 2 cintas de M1 se simulan con 1 cinta con 4 pistas de M2 (sus celdas almacenan 4-tuplas de símbolos, para los contenidos y los cabezales)

Se prueba que h pasos de M1 se pueden simular con unos h^2 pasos de M2 (retardo cuadrático)

MT no determinística (MTN)



Para un mismo par (q, w) , la máquina puede responder de más de una manera

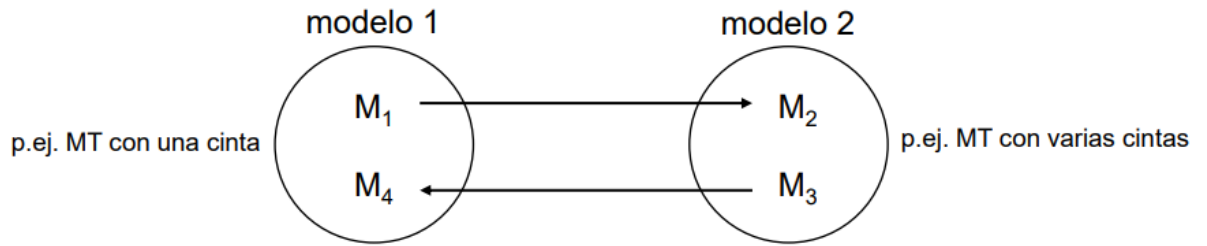
Una MTN M acepta una cadena w si y sólo si al menos una computación de M acepta w

Para simular una MTN con una MTD hay que recorrer en el peor caso todas sus computaciones. Asumiendo que toda computación hace a lo sumo h pasos, entonces ejecutar secuencialmente todas las computaciones requiere unos c^h pasos, siendo c el grado del no determinismo (retardo exponencial)

Modelos equivalentes de MT

Dos MT son equivalentes si aceptan el mismo lenguaje (es decir, si resuelven el mismo problema)

Dos modelos de MT son equivalentes si dada una MT de un modelo existe una MT equivalente del otro



Ejemplos de modelos de MT equivalentes al modelo de MT con una cinta: MT con varias cintas, MT con cintas semi-infinitas (sólo infinitas a la derecha), MT con dos cintas y un solo estado, etc.

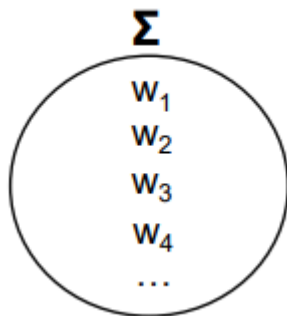
Ejemplos de modelos computacionales equivalentes al modelo de las MT: máquinas RAM (acceso directo), circuitos booleanos, lambda cálculo, funciones recursivas, gramáticas, programas C, etc.

Anexo

Acerca de los lenguajes

Σ es un alfabeto o conjunto de símbolos

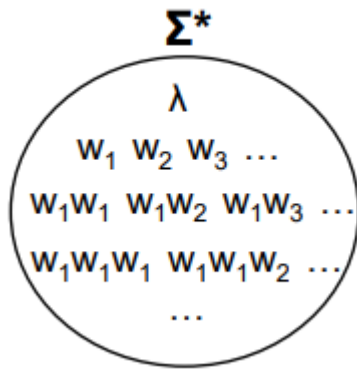
$\Sigma = \{w_1, w_2, w_3, w_4, \dots\}$



Σ^* es el lenguaje o conjunto de cadenas de símbolos generado a partir de Σ

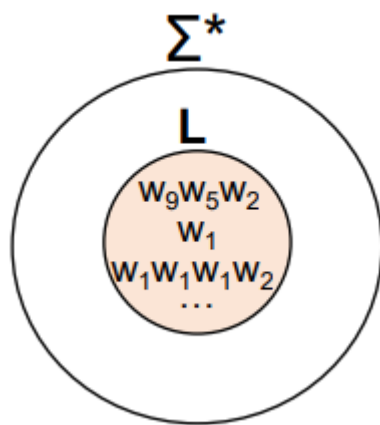
$\Sigma^* = \{\lambda, w_1, w_2, w_3, \dots, w_1w_1, w_1w_2, w_1w_3, \dots, w_1w_1w_1, w_1w_1w_2, \dots\}$

Σ^* es infinito. Sus cadenas son finitas. λ es la cadena vacía



Todo lenguaje L que consideraremos será un subconjunto de Σ^* , siendo Σ un único alfabeto que tomaremos como universal

Es decir: $L \subseteq \Sigma^*$



Por ejemplo, L puede ser el lenguaje de las cadenas que representan los grafos con un camino del vértice inicial al vértice final

Otras visiones de MT

Calculadora

Realiza un cálculo

Nos interesa además del estado final, el contenido final de la cinta

Generadora

Genera en una cinta de salida todas las cadenas que el lenguaje acepta

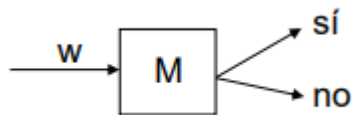
Teorema: Existe una MT M que acepta un lenguaje L si y sólo si existe una MT M' que genera el lenguaje L

Clase 2 - La jerarquía de la computabilidad

Problemas decidibles

Corresponden a lenguajes recursivos (R)

Lenguajes aceptados por MTs que siempre paran



Si $w \in L(M)$, M acepta

Si $w \notin L(M)$, M rechaza

Problema computable decidable

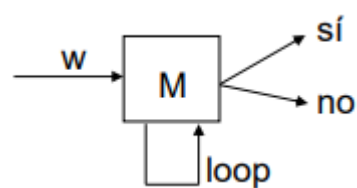
L es recursivo

R es el conjunto de estos lenguajes

Problemas computables no decidibles

Corresponden a lenguajes recursivamente enumerables (RE)

Lenguajes aceptados por MTs que a partir de algunas instancias negativas no paran



Si $w \in L(M)$, M acepta

Si $w \notin L(M)$, M rechaza o loopea

Problema computable no decidable

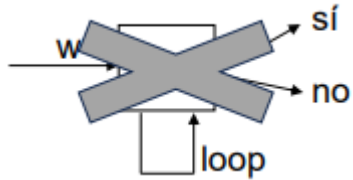
L es recursivamente enumerable

RE es el conjunto de estos lenguajes

Problemas no computables

Corresponden a lenguajes no recursivamente enumerables

Lenguajes sin MT que los acepten (a partir de algunas instancias positivas no paran)



Si $w \in L(M)$, M acepta

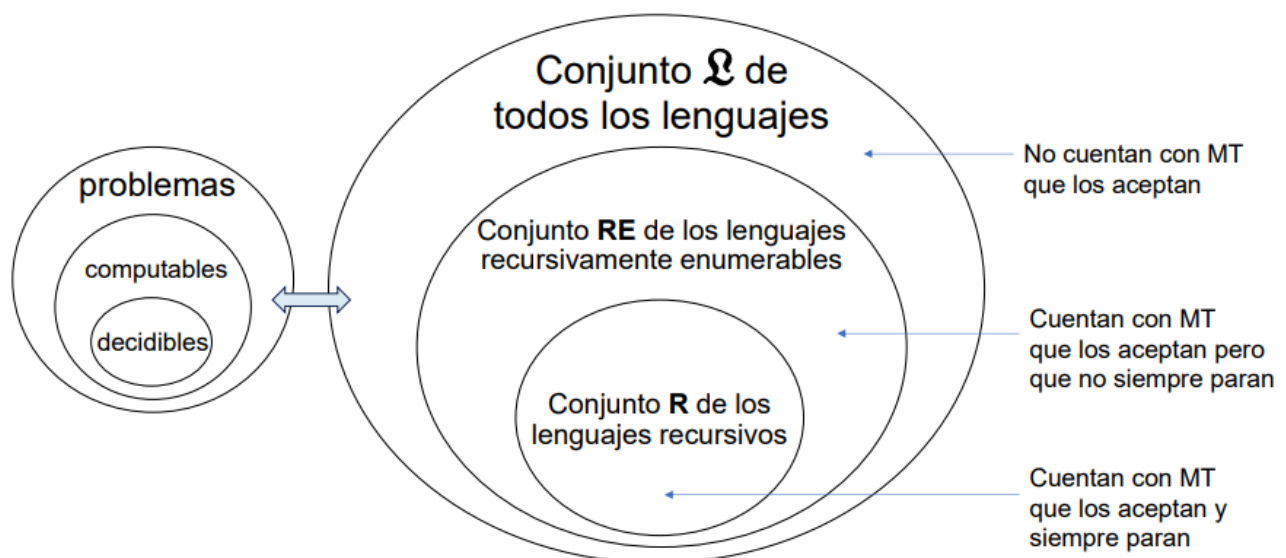
Si $w \notin L(M)$, M rechaza o loopea

Problema computable no decidable

L no es recursivamente enumerable

$\mathcal{L}$ es el conjunto de estos lenguajes

Jerarquía de la computabilidad



Definiciones

Σ

Σ es el alfabeto universal de todos los símbolos: $\Sigma = \{a, b, \dots, 1, 2, \dots, +, -, \dots\}$

Σ^*

Σ^* es el conjunto universal de todas las cadenas de símbolos de Σ : $\Sigma^* = \{\lambda, a, b, 1, \dots, aa, ab, a1, \dots, aaa, \dots\}$

$\mathcal{L}$

$\mathcal{L}$ es el conjunto universal de todos los lenguajes con alfabeto Σ : conjunto de todos los subconjuntos de Σ^*

Lenguaje recursivo

Un lenguaje L es recursivo ($L \in R$) si y sólo si existe una MT M que lo acepta y siempre para (lo decide)

Es decir, para toda cadena w de Σ^* :

- Si $w \in L$, entonces M a partir de w para en su estado q_A
- Si $w \notin L$, entonces M a partir de w para en su estado q_R

Lenguaje recursivamente enumerable

Un lenguaje L es recursivamente enumerable ($L \in RE$) si y sólo si existe una MT M que lo acepta (no necesariamente siempre para)

Es decir, para toda cadena w de Σ^* :

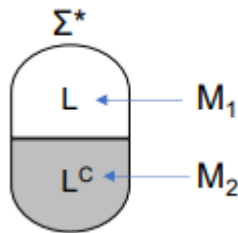
- Si $w \in L$, entonces M a partir de w para en su estado q_A
- Si $w \notin L$, entonces M a partir de w para en su estado q_R o loopea

Clase R

Propiedad 1

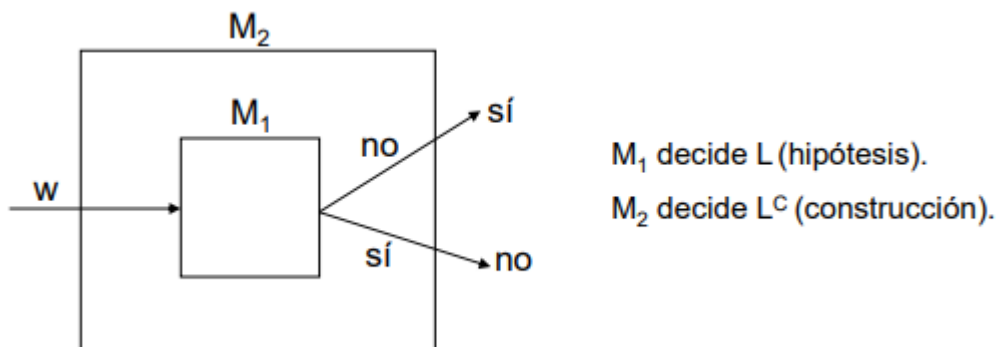
Si $L \in R$, entonces $L^c \in R$

Es decir, si existe una MT M_1 que decide L , también existe una MT M_2 que decide L^c



Prueba

Construir una MT M_2 que responda al revés que la MT M_1



Construcción de la MT M_2

Si: $M_1 = (Q, \Gamma, \delta, q_0, q_A, q_R)$

Entonces: $M_2 = (Q, \Gamma, \delta', q_0, q_A, q_R)$

Donde las funciones de transición δ y δ' de M_1 y M_2 son iguales salvo que los estados q_A y q_R están permutados

Formalmente, para todos los estados q y q' , símbolos s y s' , y movimientos d de $\{L, R, S\}$:

- Si $\delta(q, s) = (q_A, s', d)$, entonces $\delta'(q, s) = (q_R, s', d)$ *** se cambia q_A por q_R
- Si $\delta(q, s) = (q_R, s', d)$, entonces $\delta'(q, s) = (q_A, s', d)$ *** se cambia q_R por q_A
- Si $\delta(q, s) = (q', s', d)$, con $q' \neq q_A$ y $q' \neq q_R$, entonces $\delta'(q, s) = (q', s', d)$ *** los otros casos quedan igual

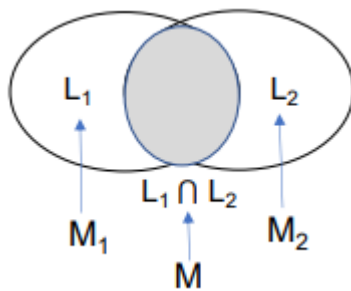
Conclusión

Si un problema es decidable, entonces también lo es el problema contrario

Propiedad 2

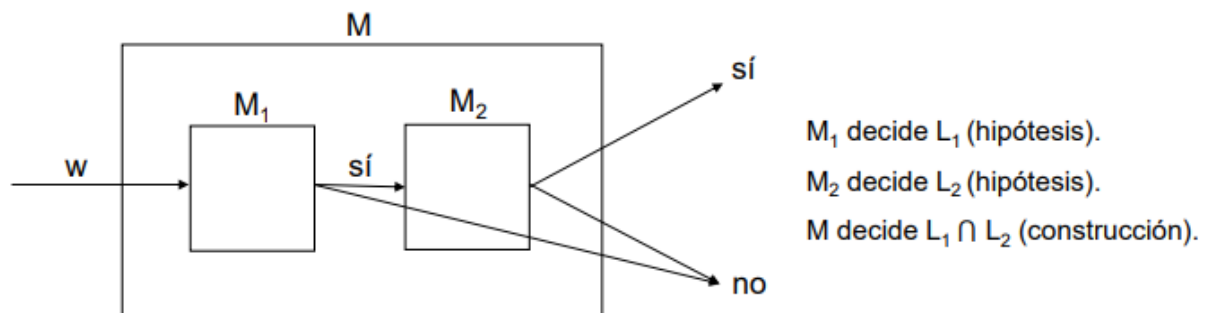
Si $L_1 \in R$ y $L_2 \in R$, entonces $L_1 \cap L_2 \in R$

Es decir, si existe una MT M_1 que decide L_1 , y existe una MT M_2 que decide L_2 , también existe una MT M que decide $L_1 \cap L_2$



Prueba

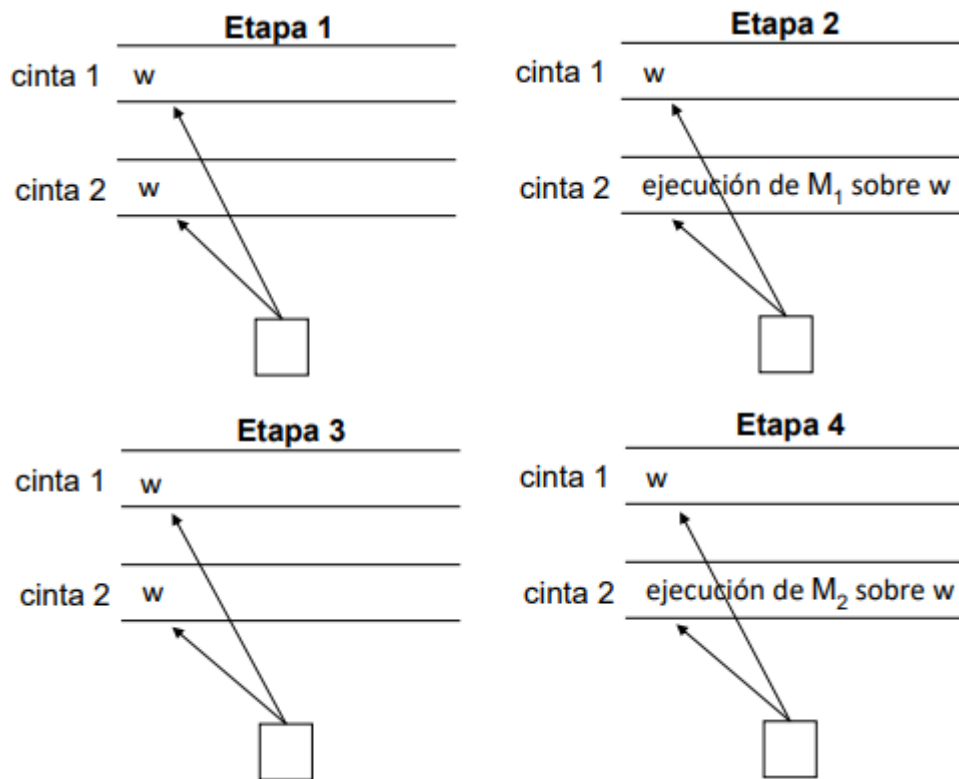
Construir una MT M que ejecute secuencialmente las MT M_1 y M_2 y acepte si y sólo si M_1 y M_2 aceptan



Construcción de la MT M

M tiene 2 cintas. Con w en la cinta 1, hace:

1. Copia w en la cinta 2
2. Ejecuta M1 sobre w en la cinta 2:
 - a. Si M1 para en qR, entonces M para en qR
3. Borra el contenido de la cinta 2 y copia de nuevo w en la cinta 2
4. Ejecuta M2 sobre w en la cinta 2:
 - a. Si M2 para en qA o en qR, entonces M para en qA o en qR (según donde haya parado M2)



Las etapas 2 y 4 pueden verse como invocaciones a rutinas, que no son más que las funciones de transición de M1 y M2

Es decir, el código de M puede verse de la siguiente manera: $\delta_M = \dots \delta_{M1} \dots \delta_{M2} \dots$

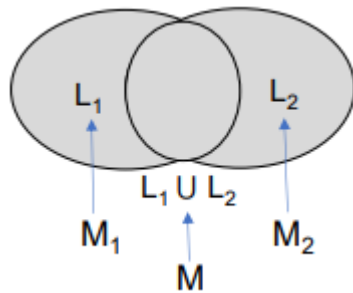
También se cumple que si $L1 \in R$ y $L2 \in R$, entonces $L1 \cup L2 \in R$

Clase RE

Propiedad 3

Si $L_1 \in RE$ y $L_2 \in RE$, entonces $L_1 \cup L_2 \in RE$

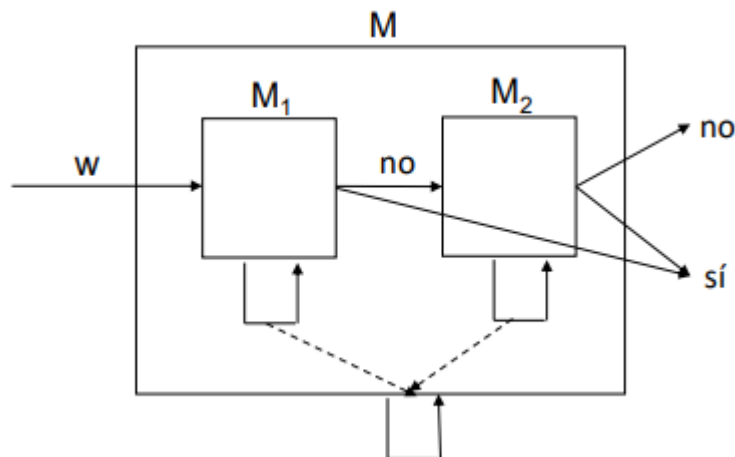
Es decir, si existe una MT M_1 que acepta L_1 , y existe una MT M_2 que acepta L_2 , también existe una MT M que acepta $L_1 \cup L_2$



Prueba

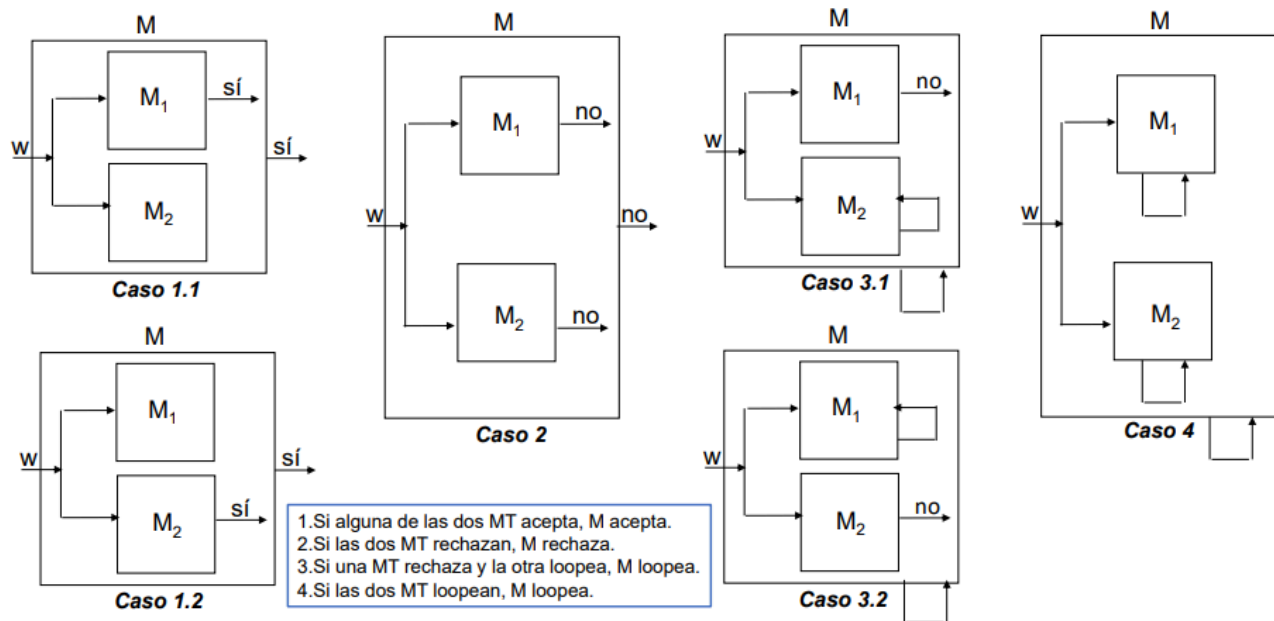
Construir una MT M que ejecute en paralelo las MT M_1 y M_2 y acepte si y sólo si M_1 o M_2 aceptan

La siguiente solución es secuencial y no es correcta (porque si M_1 loopea y M_2 acepta, M loopearía cuando debería aceptar):



La forma correcta es ejecutar “en paralelo”

“En paralelo” M_1 y M_2 = ejecutar un paso de M_1 y un paso de M_2 alternadamente



Construcción de la MT M

M tiene 3 cintas

En la cinta 1 tiene la entrada 2

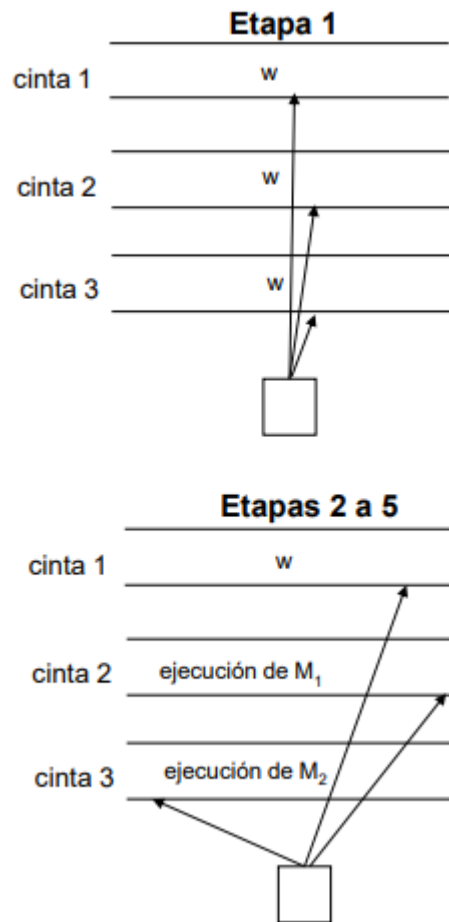
En las cintas 2 y 3 se ejecuta M_1 y M_2 respectivamente

La MT M hace:

1. Copia w de la cinta 1 a las cintas 2 y 3
2. Ejecuta un paso de M_1 en la cinta 2. Si M_1 acepta, acepta
3. Ejecuta un paso de la M_2 en la cinta 3. Si M_2 acepta, acepta
4. Si M_1 y M_2 rechazan, rechaza
5. Vuelve al paso 2

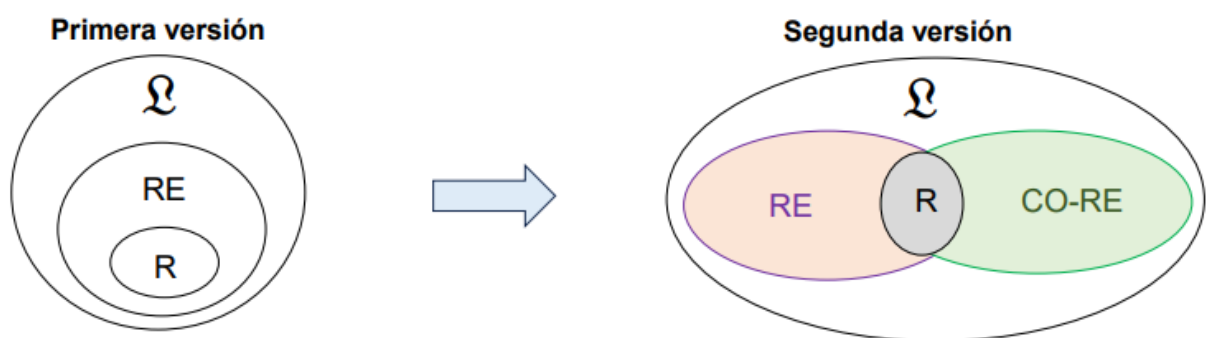
M acepta, rechaza o no para

En cada iteración memoriza los estados y posiciones de las 2 ejecuciones



También se cumple que si $L_1 \in RE$ y $L_2 \in RE$, entonces $L_1 \cap L_2 \in RE$

Segunda versión de la jerarquía de la computabilidad



El conjunto CO-RE tiene los complementos de los lenguajes del conjunto RE ($L \in RE$ si y sólo si $L^c \in CO-RE$)

Claramente se cumple $R \subseteq RE \cap CO-RE$:

- $R \subseteq RE$ por definición
- $R \subseteq CO-RE$ porque:
 - Si $L \in R$, entonces $L^c \in R$, por lo que $L^c \in RE$, y así por definición:
 $L \in CO-RE$

Propiedad 4

$$RE \cap CO-RE = R$$

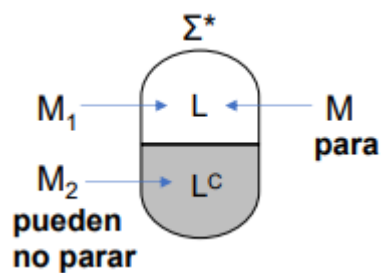
Solo falta probar la inclusión de $RE \cap CO-RE \subseteq R$

Hay que probar que si $L \in RE \cap CO-RE$, entonces $L \in R$

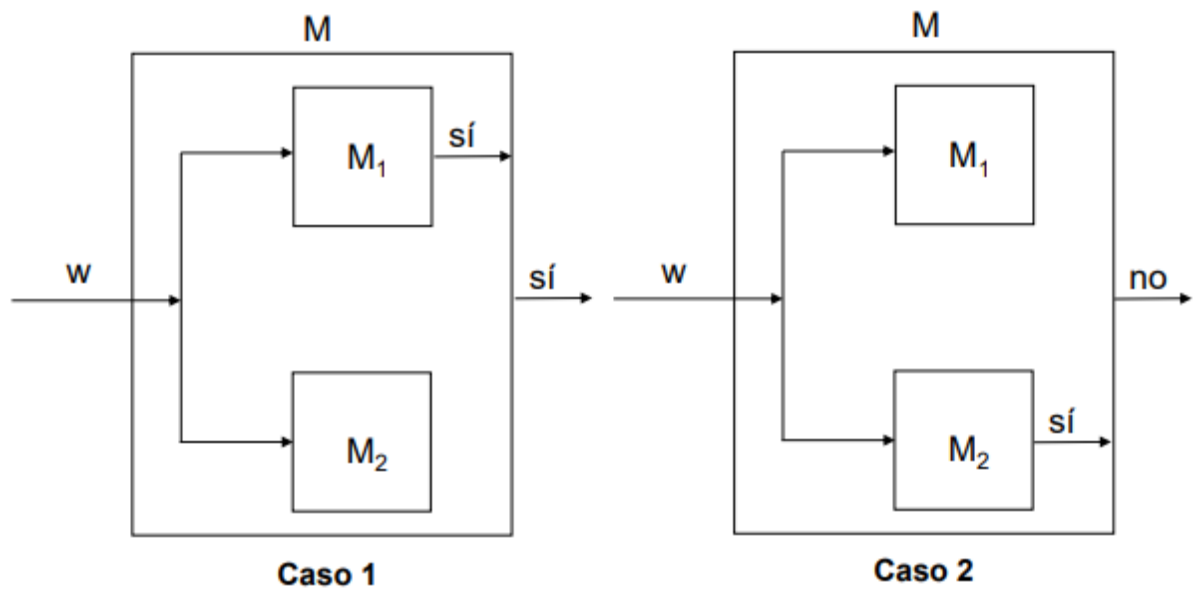
Es decir, que si $L \in RE$ y $L \in CO-RE$, entonces $L \in R$

Es decir, que si $L \in RE$ y $L^c \in RE$, entonces $L \in R$

Hay que probar, entonces, que si existe una MT M_1 que acepta L , y existe una MT M_2 que acepta L^c , también existe una MT M que decide L



Prueba



M ejecuta en paralelo M_1 y M_2

Si M_1 acepta w , M acepta w

Si M_2 acepta w , M rechaza w

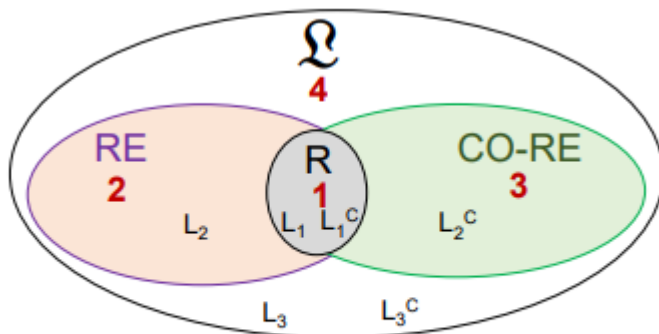
Entonces, cualquiera sea w , M_1 la acepta o M_2 la acepta

¿Por qué? Porque toda cadena w pertenece a L o a L^c , entonces, o acepta M_1 o acepta M_2

Conclusión

Si un problema y el problema contrario son computables, entonces ambos son decidibles

Las cuatro regiones de la jerarquía de la computabilidad



Las cuatro regiones de la jerarquía,
con grado de dificultad creciente.

Región 1 (lenguajes con MT que siempre paran)

Conjunto R

Si L está en R , entonces L^C está en R

Región 2 (lenguajes con MT)

Conjunto $RE - R$

Si L está en RE , entonces L^C está en $CO-RE$

Región 3 (lenguajes sin MT, con complementos con MT)

Conjunto $CO-RE - R$

Si L está en $CO-RE$, entonces L^C está en RE

Región 4 (lenguajes sin MT, con complementos sin MT)

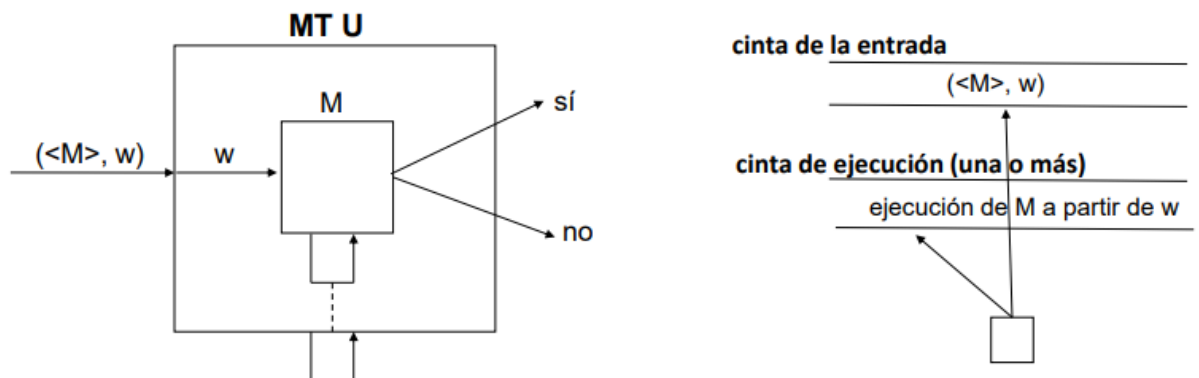
Conjunto $\mathcal{Q} - (RE \cup CO-RE)$

Si L está en $\mathcal{Q} - (RE \cup CO-RE)$, entonces L^C está en $\mathcal{Q} - (RE \cup CO-RE)$

Clase 3 - Indecibilidad

Máquina de Turing universal

Una máquina de Turing universal (MT U) es una máquina de Turing capaz de ejecutar cualquier otra (noción de programa almacenado, Turing 1936). El esquema más general es:



La MT U recibe como entrada una MT M (codificada mediante una cadena) y una cadena w, y ejecuta M a partir de w

Codificación de una máquina de Turing y una cadena de entrada

Hay varias maneras. P.ej., usando números en binario (cadenas de 1 y 0). Entre otras características:

- El código del estado inicial q_0 es 1, el código del estado q_A es 2, y el código del estado q_R es 3
- El código del símbolo blanco es 1
- Los movimientos R, L y S se codifican con 1, 2 y 3, respectivamente

Ejemplo:

Si M es:

$$\delta(q_0, s) = (q_0, t, L),$$

$$\delta(q_0, t) = (q_0, s, L),$$

$$\delta(q_0, B) = (q_A, B, S)$$

Y el código del símbolo s es 2 y el código del símbolo t es 3

Queda $\langle M \rangle = (1,2,1,3,2), (1,3,1,2,2), (1,1,2,1,3)$

Y si la cadena w es: st, queda: $(\langle M \rangle, w) = (1,2,1,3,2),(1,3,1,2,2),(1,1,2,1,3)\#2,3$ (para simplificar la escritura, usamos w en lugar de $\langle w \rangle$)

Enumeración de las máquinas de Turing

Codificar las máquinas de Turing permite enumerarlas, por ejemplo en el orden canónico:

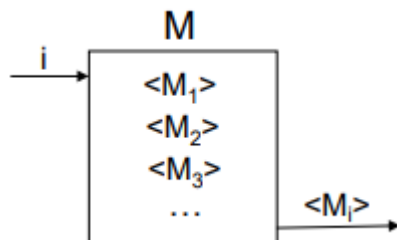
1. Los códigos se ordenan de menor a mayor longitud
2. Los códigos con longitudes iguales se ordenan según el orden alfanumérico

Por ejemplo, las primeras cadenas según el orden canónico, con ceros, unos, paréntesis y comas, son:

1. λ
2. $0\ 1\ (\) \ ,$
3. $00\ 01\ 0(0)\ 0, 10\ 11\ 1(1)\ 1, (0\ (1\ ((\ \dots$
4. $000\ 001\ 00(00)\ 00, 010\ 011\ 01(01)\ 01, 0(0\ 0(1\ 0((\ \dots \text{ etc.}$

La siguiente MT M obtiene la MT i -ésima (MT M_i) según el orden canónico. Dado i , la MT M hace:

1. Hace $n := 0$
2. Genera la siguiente cadena v según el orden canónico
3. Valida que v sea el código de una MT. Si no lo es, vuelve al paso 2
4.
 - a. Si $n = i$, devuelve v (v es el código de la MT M_i) y para
 - b. Si $n \neq i$, hace $n := n + 1$ y vuelve al paso 2



Prueba de que $R \subset RE \subset \mathcal{Q}$

La siguiente tabla T representa el comportamiento de todas las MT M con respecto a todas las cadenas w:

todas las cadenas							todas las MT
	T	w ₀	w ₁	w ₂	w ₃	w ₄	
fila 0	M ₀	1	0	1	1	1	
fila 1	M ₁	1	0	0	1	0	
fila 2	M ₂	0	0	1	0	1	
fila 3	M ₃	0	1	1	1	1	
fila 4	M ₄	0	1	1	1	0	
							

$T(M_i, w_j) = 1$ si M_i acepta w_j ; $T(M_i, w_j) = 0$ si M_i rechaza w_j (los valores son de ejemplo)

- La fila 0 = (1, 0, 1, 1, 1, ...), representa el lenguaje $L(M_0) = \{w_0, w_2, w_3, w_4, \dots\}$
- La fila 1 = (1, 0, 0, 1, 0, ...), representa el lenguaje $L(M_1) = \{w_0, w_3, \dots\}$
- La fila 2 = (0, 0, 1, 0, 1, ...), representa el lenguaje $L(M_2) = \{w_2, w_4, \dots\}$
- Etc.

Por lo tanto, las filas representan todos los lenguajes aceptados por MT, es decir el conjunto RE

Construimos en particular la diagonal de la tabla T:

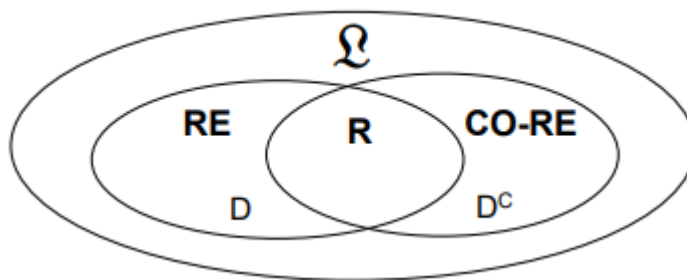
	T	w ₀	w ₁	w ₂	w ₃	w ₄	
fila 0	M ₀	1	0	1	1	1	
fila 1	M ₁	1	0	0	1	0	
fila 2	M ₂	0	0	1	0	1	
fila 3	M ₃	0	1	1	1	1	
fila 4	M ₄	0	1	1	1	0	
							

La diagonal es (1, 0, 1, 1, 0, ...), y representa el lenguaje $D = \{w_i \mid M_i \text{ acepta } w_i\}$. En el ejemplo: $D = \{w_0, w_2, w_3, \dots\}$

Y la diagonal con los 1 y 0 invertidos es $(0, 1, 0, 0, 1, \dots)$, y representa el lenguaje $D^c = \{w_i \mid M_i \text{ rechaza } w_i\}$. En el ejemplo: $D^c = \{w_1, w_4, \dots\}$

Vamos a probar:

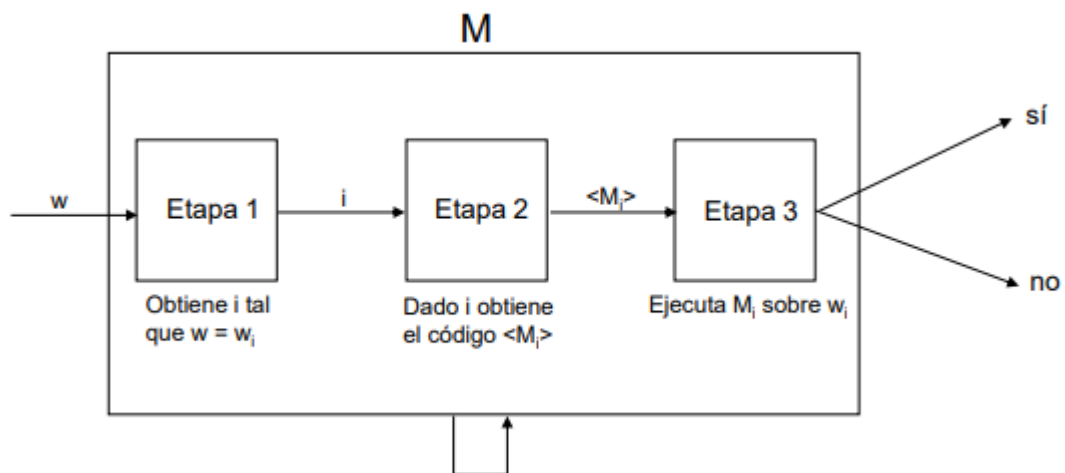
1. D está en RE
2. D^c no está en RE
3. Y por lo tanto: D no está en R



Probamos que D está en RE

La siguiente MT M acepta D . Dada una entrada w , M hace:

1. Encuentra i tal que $w = w_i$ en el orden canónico (genera cadenas en el orden canónico hasta encontrar w)
2. Encuentra el código $\langle M_i \rangle$ de la MT M_i (genera códigos de MT en el orden canónico hasta llegar a $\langle M_i \rangle$)
3. Ejecuta M_i sobre w_i y acepta si y sólo si M_i acepta



Por lo tanto, se cumple $D = \{w_i \mid M_i \text{ acepta } w_i\} \in \text{RE}$

Probamos que D^c no está en RE

Vimos que la diagonal con los 1 y 0 invertidos es $(0, 1, 0, 0, 1, \dots)$, y que representa $D^c = \{w_i \mid M_i \text{ rechaza } w_i\}$

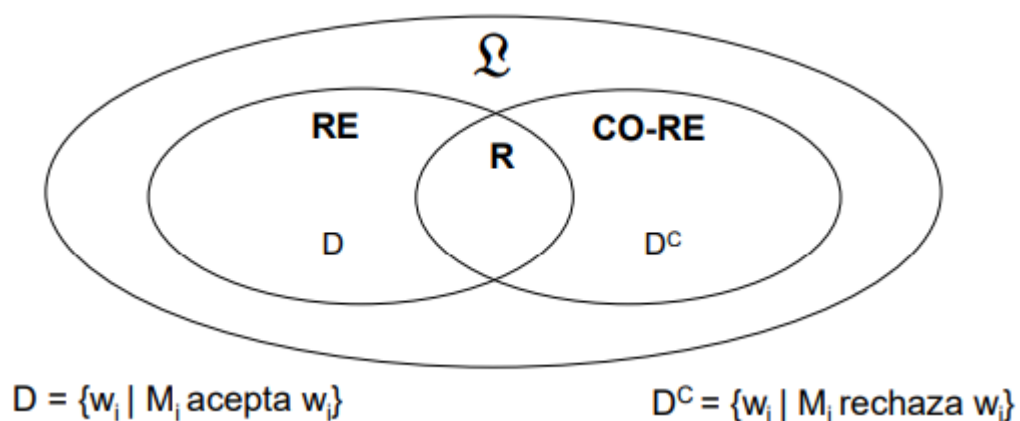
Notar que dicha diagonal es diferente de todas las filas:

- Es diferente de la fila 0 = $(1, 0, 1, 1, 1, \dots)$, que representa el lenguaje $L(M_0)$, en el primer elemento
- Es diferente de la fila 1 = $(1, 0, 0, 1, 0, \dots)$, que representa el lenguaje $L(M_1)$, en el segundo elemento
- Es diferente de la fila 3 = $(0, 0, 1, 0, 1, \dots)$, que representa el lenguaje $L(M_2)$, en el tercer elemento
- Etc.

Y como las filas representan todos los lenguajes de RE, D^c es diferente de todos los lenguajes de RE

Por lo tanto, se cumple $D^c = \{w_i \mid M_i \text{ rechaza } w_i\} \notin \text{RE}$

Así encontramos dos primeros lenguajes fuera de R (el lenguaje D^c por diagonalización):



A partir de estos primeros lenguajes se pueden encontrar otros lenguajes fuera de R y RE, con un método más sencillo, la reducción (lo veremos en la clase siguiente)

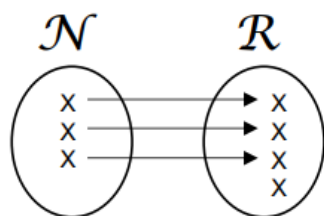
La diagonalización es muy útil para encontrar primeros lenguajes en un conjunto

Consiste básicamente en encontrar un lenguaje separador. Por ejemplo, el lenguaje D^c actúa como separador entre los conjuntos RE y L (no pertenece a RE y pertenece a L)

Anexo

$$|\mathcal{Q}| = |\mathcal{R}| > |\mathcal{RE}| = |\mathcal{N}|$$

Se prueba por diagonalización que el conjunto $\mathcal{R}$ de los números reales es más grande que el conjunto $\mathcal{N}$ de los números naturales (enteros positivos), es decir que los números reales no se pueden enumerar



Hay más números reales que números naturales.

En símbolos: $|\mathcal{R}| > |\mathcal{N}|$.

($|C|$ denota el tamaño o la cardinalidad del conjunto C).

0,1287...

0,8550...

0,1380...

0,2751...

etc.

Y sea el número coloreado 0,1581... Sumando 1 a sus decimales obtenemos 0,2692...

Veamos que este número entre el 0 y el 1 no está en la enumeración (contradicción de acuerdo a lo que asumimos):

- 0,2692... difiere de 0,1287... en el primer decimal
- 0,2692... difiere de 0,8550... en el segundo decimal
- 0,2692... difiere de 0,1380... en el tercer decimal
- Etc.

Por lo tanto, $|\mathcal{R}| > |\mathcal{N}|$

Además, el conjunto R de los números reales es mucho más grande que el conjunto N de los números naturales

Y como $|R| = |\mathcal{Q}|$ y $|N| = |RE|$, entonces $\mathcal{Q}$ es mucho más grande que RE

¡Hay muchos más problemas que problemas computables!

$|N| = |RE|$ porque las MT se pueden enumerar, y entonces también los lenguajes de RE se pueden enumerar

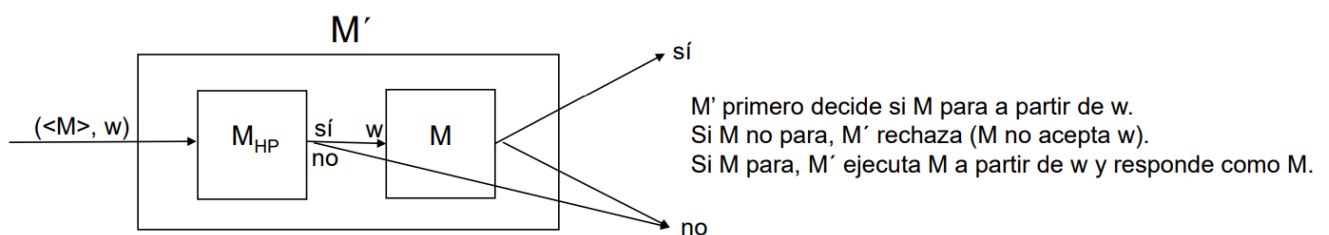
$|R| = |\mathcal{Q}|$ porque:

- Se prueba que $|R| = |P(N)|$, siendo $P(N)$ el conjunto de partes de N (el conjunto de todos sus subconjuntos)
- $|N| = |\Sigma^*|$, porque las cadenas de Σ^* se pueden enumerar. Así, $|P(N)| = |P(\Sigma^*)|$
- $P(\Sigma^*) = \mathcal{Q}$, porque $\mathcal{Q}$ es el conjunto de todos los subconjuntos de Σ^* . Así, $|P(\Sigma^*)| = |\mathcal{Q}|$
- En consecuencia: $|R| = |P(N)| = |P(\Sigma^*)| = |\mathcal{Q}|$

Halting Problem

Se puede demostrar que HP pertenece a $RE - R$, por el absurdo:

1. Si HP fuera recursivo, entonces, todo lenguaje de RE lo sería
2. Sea M_{HP} una MT que decide HP y M una MT que reconoce algún lenguaje L de RE . La siguiente MT M' decide L :



Si $\langle M \rangle$ no se detiene a partir de w , entonces, M_{HP} y M' rechazan (se detiene)

Si $\langle M \rangle$ se detiene a partir de w , entonces M_{HP} acepta y pasa w a M , entonces, como sabemos que M se detiene a partir de w , cuando M acepte o rechace, así lo hará M' , por lo que M' decide a $L(M)$, es decir, cualquier lenguaje con una MT que lo acepte (que esté en RE), tendría una MT que lo decida ($R = RE$), cosa que ya probamos falsa

Clase 4 - Las reducciones y otros temas de computabilidad

Reducción de lenguajes

Es un método para probar no pertenencia, en nuestro caso, a R y RE

Dados dos lenguajes L_1 y L_2 , una reducción de L_1 a L_2 es una función total computable $f : \Sigma^* \rightarrow \Sigma^*$ (función definida para todas las cadenas de Σ^* y computable por una MT M_f), que para toda cadena w :

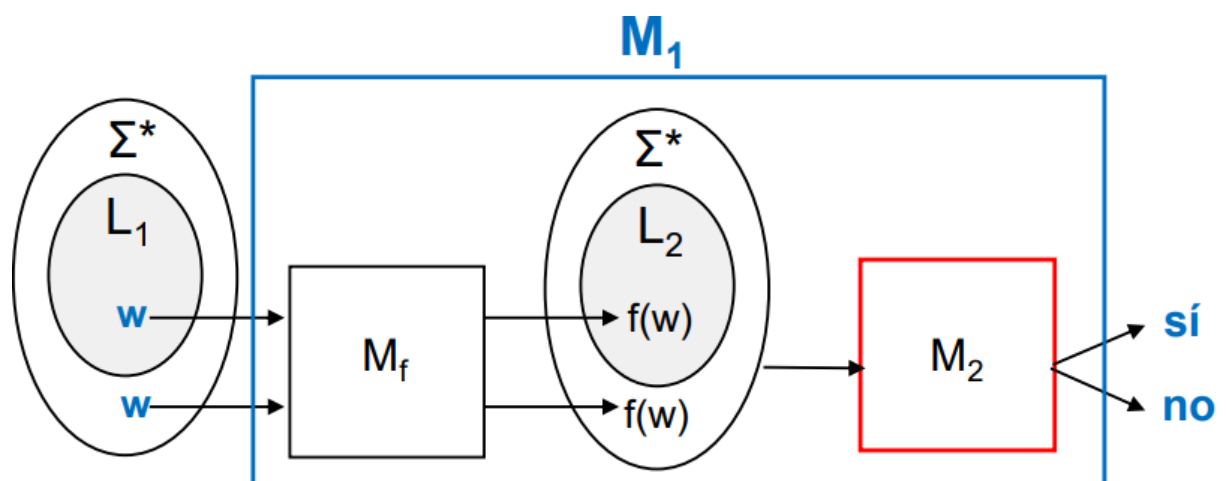
- si $w \in L_1$, entonces $f(w) \in L_2$
- si $w \notin L_1$, entonces $f(w) \notin L_2$

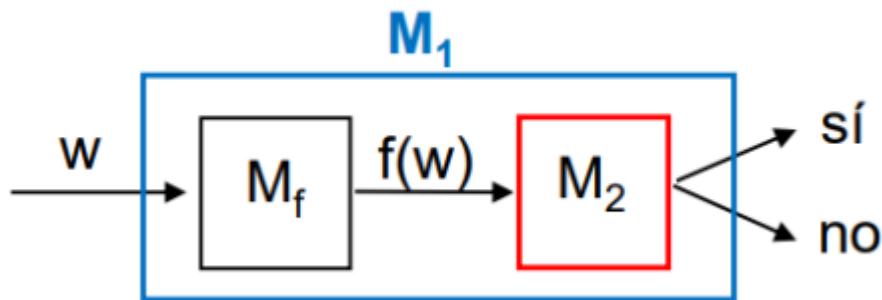
Objetivo: reducir (relacionar) el lenguaje L_1 al lenguaje L_2 , con la idea de construir una MT M_1 que acepte L_1 utilizando una MT M_2 ya conocida que acepta L_2 , en lugar de construir M_1 de cero

¿A qué técnica de programación se asemeja esto? $\rightarrow$ A la invocación a una subrutina

La notación $L_1 \leq L_2$ expresa que existe una reducción de L_1 a L_2

Caso 1: $L_2 \in R$





Si M_2 decide L_2 , entonces M_1 decide L_1

Si $w \in L_1$, entonces $f(w) \in L_2$, entonces M_2 acepta $f(w)$, entonces M_1 acepta w

Si $w \notin L_1$, entonces $f(w) \notin L_2$, entonces M_2 rechaza $f(w)$, entonces M_1 rechaza w

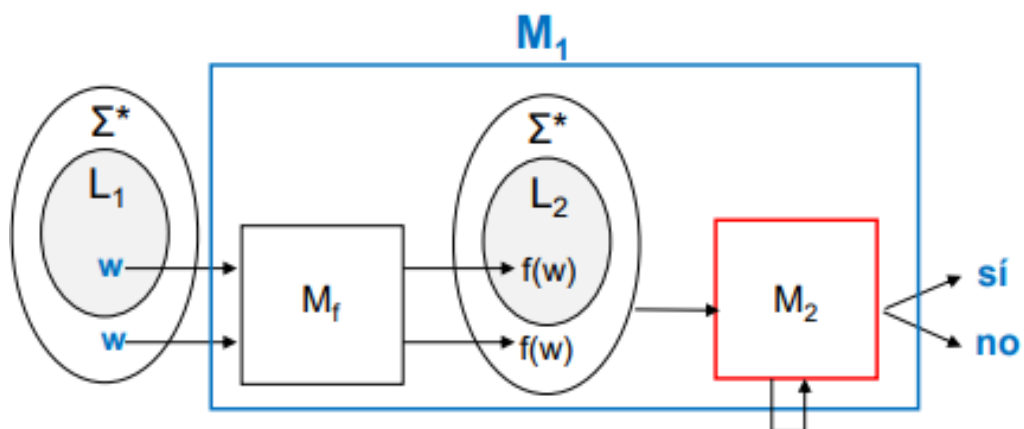
En lugar de construir de cero una MT M_1 para decidir L_1 , se la puede construir a partir de una MT M_2 conocida que decide L_2 (noción de invocación a una subrutina en programación)

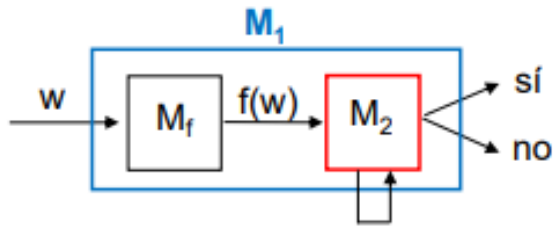
Formalmente:

- si $L_1 \leq L_2$, entonces $L_2 \in R \rightarrow L_1 \in R$
- O lo mismo: si $L_1 \leq L_2$, entonces $L_1 \notin R \rightarrow L_2 \notin R$

Encontrando una reducción de un lenguaje L_1 que no está en R a un lenguaje L_2 , se prueba que L_2 tampoco está en R

Caso 2: $L_2 \in RE$





Si M_2 acepta L_2 , entonces M_1 acepta L_1

Si $w \in L_1$, entonces $f(w) \in L_2$, entonces M_2 acepta $f(w)$, entonces M_1 acepta w

Si $w \notin L_1$, entonces $f(w) \notin L_2$, entonces M_2 rechaza $f(w)$ (puede looppear), entonces M_1 rechaza w (puede looppear)

En lugar de construir de cero una MT M_1 para aceptar L_1 , se la puede construir a partir de una MT M_2 conocida que acepta L_2 (noción de invocación a una subrutina en programación)

Formalmente:

- si $L_1 \leq L_2$, entonces $L_2 \in RE \rightarrow L_1 \in RE$
- O lo mismo: si $L_1 \leq L_2$, entonces $L_1 \notin RE \rightarrow L_2 \notin RE$

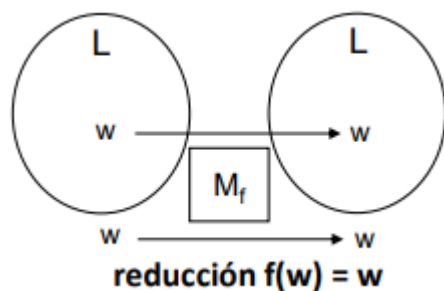
Encontrando una reducción de un lenguaje L_1 que no está en R a un lenguaje L_2 , se prueba que L_2 tampoco está en R

Propiedades de las reducciones

Reflexividad

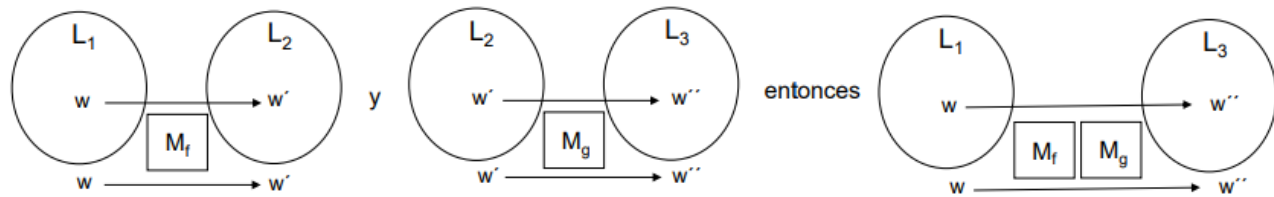
Para todo lenguaje L se cumple $L \leq L$

La reducción es la función identidad (para cada cadena se retorna la misma cadena)



Transitividad

Si $L_1 \leq L_2$ y $L_2 \leq L_3$, entonces $L_1 \leq L_3$



La reducción es la composición de las dos reducciones

Otra propiedad

$L_1 \leq L_2$ si y sólo si $L_1^C \leq L_2^C$

La reducción es la misma

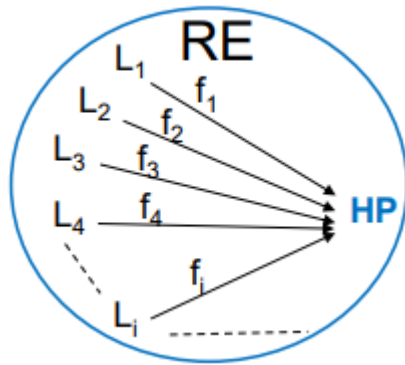


No se cumple la simetría

$L_1 \leq L_2$ no implica $L_2 \leq L_1$ (la propiedad de simetría sólo se cumple en R)

Otra manera de probar que HP está entre los lenguajes más difíciles de RE

Se prueba que todo lenguaje L de RE cumple $L \leq HP$



HP es tan o más difícil que cualquier lenguaje de RE

Decidiendo HP se decide cualquier lenguaje L_i

Así, si HP estuviera en R, se cumpliría $R = RE$

HP identifica la dificultad de la clase RE, es RE-completo

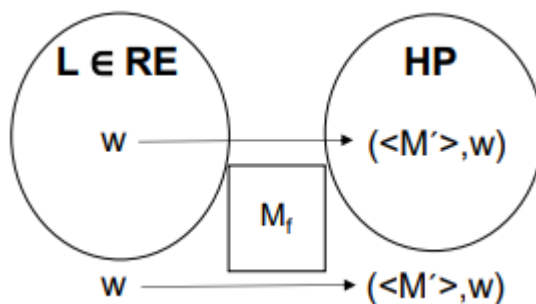
Idea general de la prueba

Sea M una MT que acepta L

M' es como M , salvo que desde todo qR de M se incluye en M' un loop

De esta manera:

- $w \in L \rightarrow M$ acepta $w \rightarrow M'$ para a partir de w
- $w \notin L \rightarrow M$ rechaza $w \rightarrow M'$ no para a partir de w



Anexo

Máquinas de Turing restringidas

Autómatas finitos

Una cinta de sólo lectura

Sólo se mueve a la derecha

Conjunto F de estados finales

Cuando alcanza el símbolo B (blanco), el AF para, y acepta si y sólo si el estado alcanzado es un estado final

El AF constituye un tipo de algoritmo muy utilizado

Por ejemplo:

- Para el análisis sintáctico a nivel palabra de los compiladores (if, then, else, while, x, 10, =, +, etc)
- Para las inspecciones de código en el control de calidad del software

Ejemplo

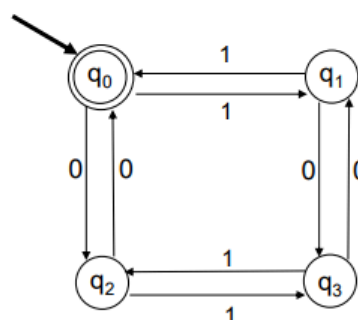
$Q = \{q_0, q_1, q_2, q_3\}$, $\Sigma = \{0, 1\}$, Estado inicial q_0 , $F = \{q_0\}$

- | | |
|---------------------------|---------------------------|
| 1. $\delta(q_0, 1) = q_1$ | 2. $\delta(q_1, 1) = q_0$ |
| 3. $\delta(q_1, 0) = q_3$ | 4. $\delta(q_3, 0) = q_1$ |
| 5. $\delta(q_3, 1) = q_2$ | 6. $\delta(q_2, 1) = q_3$ |
| 7. $\delta(q_2, 0) = q_0$ | 8. $\delta(q_0, 0) = q_2$ |

Diagrama de transición de estados

La ejecución arranca desde la flecha.

Los estados con doble contorno son los estados finales.



El AF descrito acepta todas las cadenas de 1 y 0 con una cantidad par de 1 y una cantidad par de 0.

Características de los AF

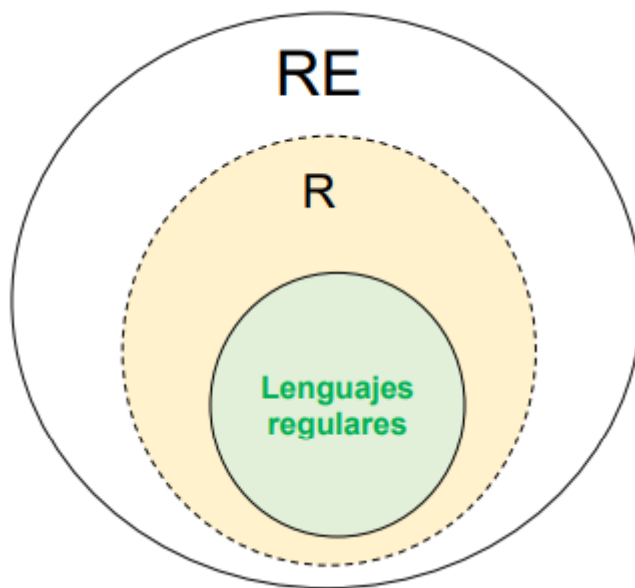
Los AF siempre paran

Aceptan un tipo limitado de cadenas (no tienen memoria). No pueden aceptar cadenas con igual cantidad de a y b, no pueden chequear si una cadena es un palíndromo, etc.. En general, no pueden calcular

Los lenguajes que aceptan los AF se llaman regulares o de tipo 3

A diferencia de las MT generales, siempre se puede decidir si un AF:

- Acepta una cadena w
- Acepta el lenguaje $\emptyset$
- Acepta el lenguaje Σ^*
- Es equivalente a otro
- Etc.



Autómatas con pila

Una primera cinta de entrada de sólo lectura

Una segunda cinta de lectura/escritura que se comporta como una pila

En un paso se pueden procesar las dos cintas

En la cinta de entrada siempre se va a la derecha

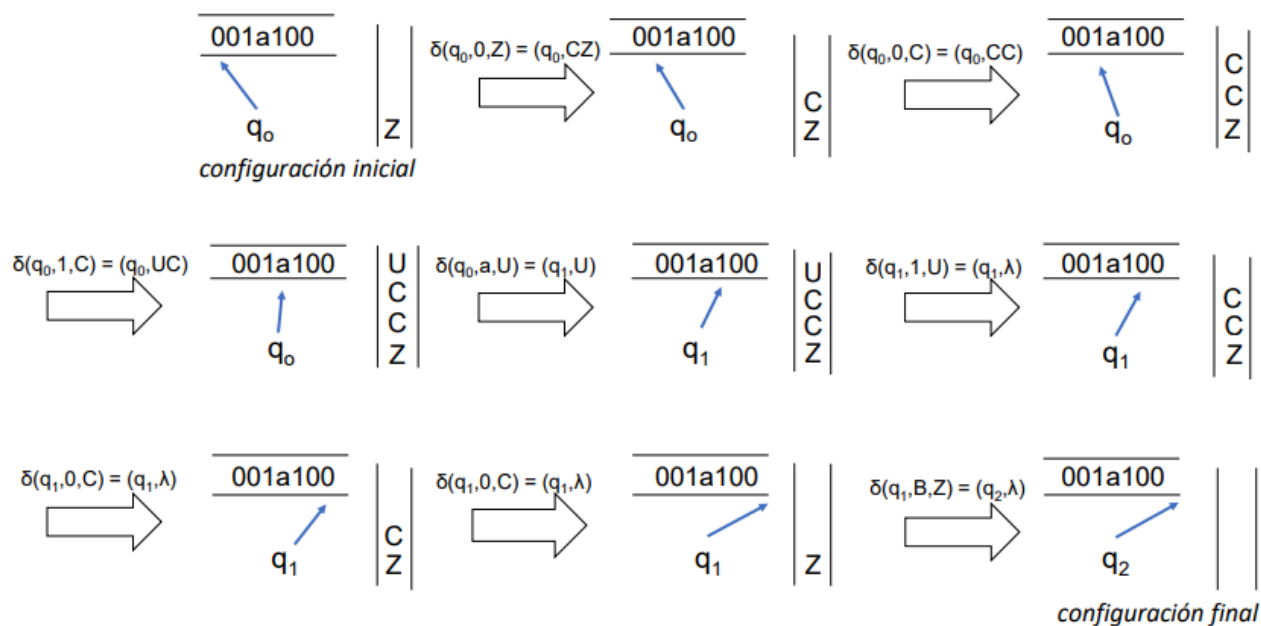
Cuando alcanza el símbolo B (blanco) en la cinta de entrada, el AP para y acepta si y sólo si la pila está vacía

Problemas típicos que resuelve un AP:

- Análisis sintáctico a nivel instrucción de los compiladores
- Evaluación de expresiones en la ejecución de programas

Ejemplo. Reconocimiento de palíndromos waw^R , tales que w tiene símbolos 0 y 1 y w^R es la inversa de w .

Supongamos la entrada **001a100**:



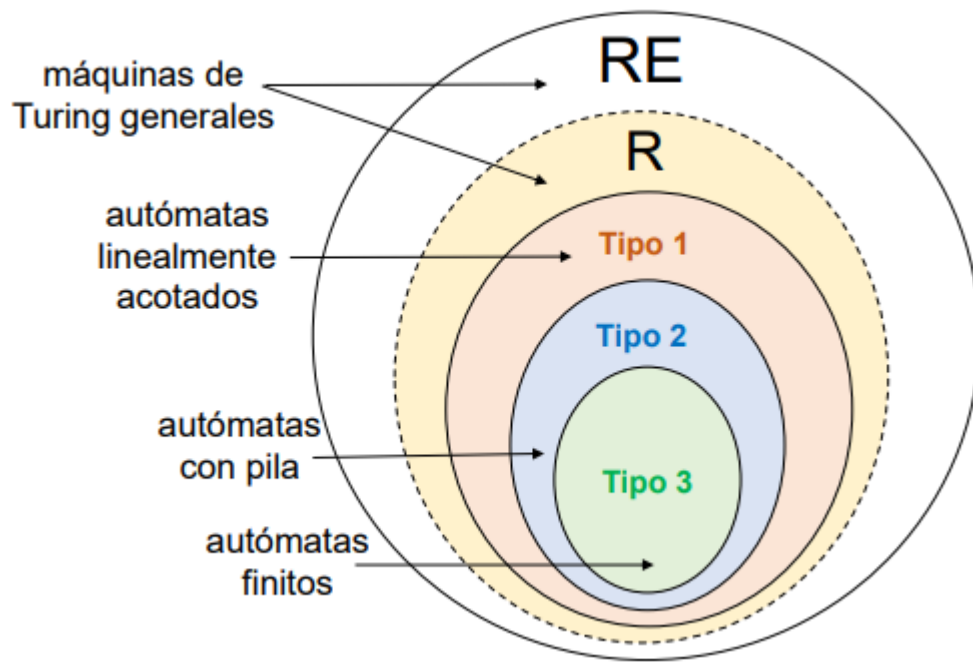
Características de los AP

Los AP siempre paran

Los lenguajes que aceptan se llaman libres de contexto o de tipo 2

A diferencia de las MT generales, siempre se puede decidir si un AP:

- Acepta una cadena w
- Acepta el lenguaje
- Etc.



Jerarquía de los lenguajes de Chomsky

De tipo 0 (recursivamente enumerables)

De tipo 1 (sensibles al contexto)

De tipo 2 (libres de contexto)

De tipo 3 (regulares)

Enumeración de los lenguajes recursivamente enumerables

Todo lenguaje L de RE se puede enumerar

Sea M_1 una MT que acepta L . Vamos a construir una MT M_2 que genera L :

1. Hacer $n := 1$
2. Generar todas las cadenas de longitud a lo sumo n en el orden canónico
3. Por cada cadena generada ejecutar a lo sumo n pasos de la MT M_1 . Si M_1 acepta, imprimir la cadena
4. Hacer $n := n + 1$ y volver al paso 2

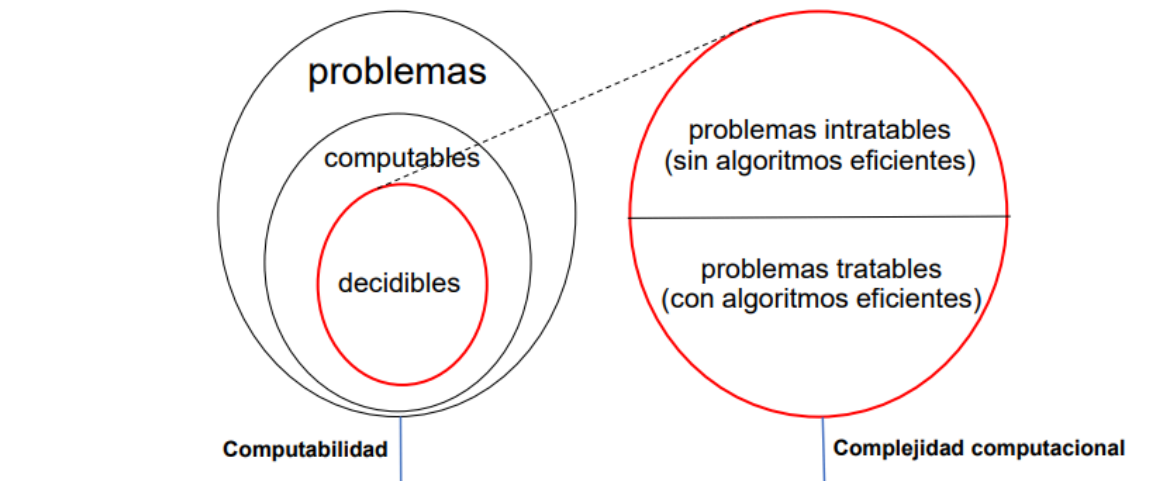
Todo lenguaje L de R se puede enumerar en el orden canónico

Sea M_1 una MT que decide L . Vamos a construir una MT M_2 que genera L en el orden canónico:

1. Generar la primera cadena en el orden canónico
2. Ejecutar M_1 sobre la cadena generada. Si acepta, imprimirla
3. Generar la siguiente cadena en el orden canónico y volver al paso 2

Clase 5 - Tiempo polinomial y no polinomial

Los problemas decidibles se separan en intratables (sin algoritmos eficientes) y tratables (con algoritmos eficientes)



Métricas de complejidad computacional más utilizadas:

- Tiempo = cantidad de pasos ejecutados por una MT
- Espacio = cantidad de celdas ocupadas por una MT

Complejidad temporal

Una MT M tarda más (hace más pasos) a medida que sus cadenas de entrada w son más grandes

Por eso el tiempo de M no se mide en términos absolutos sino con funciones temporales $T(n)$, que se definen en términos del tamaño $|w|$ de w (se usa $n = |w|$)

Ejemplos de funciones temporales $T(n)$:

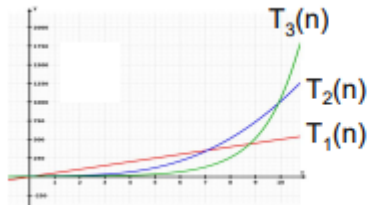
- $5n + 8$, $3n^2$, $3n^3 + n^2 + 25n$, etc. polinomiales o $\text{poly}(n)$ - n como factor
- $20^{\log 2n}$, $12^n + 10n + 5$, 6^{n^5} , etc. exponenciales o $\exp(n)$ - n como exponente
- Otras: 2 elevado a la 2^n (doble exponencial), 2 elevado a la 2 elevado a la 2^n (triple exponencial), etc.

Una función $T_1(n)$ es del orden de una función $T_2(n)$, que se anota así: $T_1(n) = O(T_2(n))$, si y sólo si para todo $n \geq n_0$ se cumple $T_1(n) \leq c \cdot T_2(n)$, con $c > 0$

Por ejemplo:

- $5n^3 + 8n + 25 = O(n^3)$

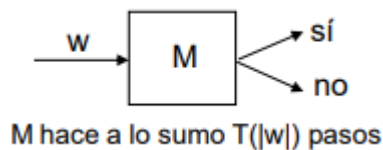
- $n^2 = O(n^3)$
- $n^3 = O(2^n)$
- $n \cdot \log_2 n = O(n^2)$



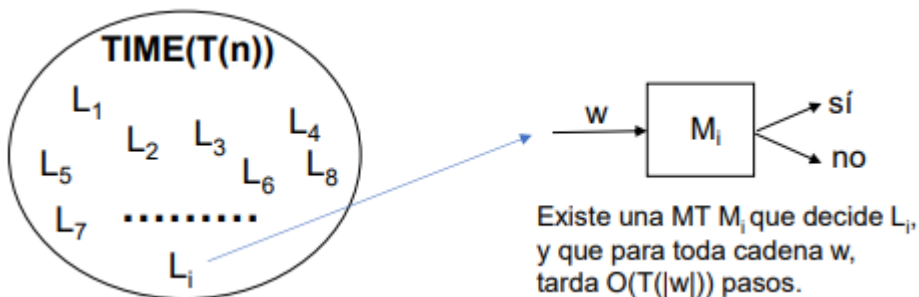
$T_i(n) = O(T_j(n))$, con $i < j$
 Cuando n tiende a infinito,
 $T_i(n)$ se mantiene por debajo de $T_j(n)$

Usar funciones $O(T(n))$ por funciones $T(n)$ permite manejar un nivel de abstracción adecuado (tiempo lineal, cuadrático, polinomial, cuasipolinomial, subexponencial, exponencial, doble exponencial, etc)

Una MT M tarda tiempo $T(n)$, si y sólo si a partir de toda entrada w , con $|w| = n$, M hace a lo sumo $T(n)$ pasos



Un lenguaje $L \in \text{TIME}(T(n))$ si y sólo si existe una MT M que lo decide en tiempo $O(T(n))$

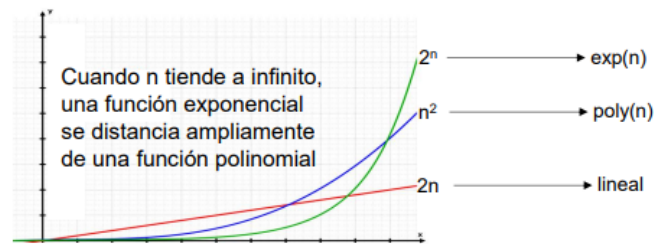


Se considera el tiempo máximo (procesar cualquier cadena w consume a lo sumo $T(|w|)$ pasos)

Otros criterios son el tiempo promedio y el tiempo mínimo, en general muy difíciles de calcular (a pesar de buenos avances, al día de hoy se conocen aún pocos valores de este tipo)

Convención, respaldada por las matemáticas y la experiencia: tiempo tratable = tiempo $\text{poly}(n)$

n	2n	n ²	2 ⁿ
0	0	0	1
1	2	1	2
2	4	4	4
3	6	9	8
4	8	16	16
5	10	25	32
6	12	36	64
7	14	49	128
8	16	64	256
9	18	81	512
10	20	100	1024

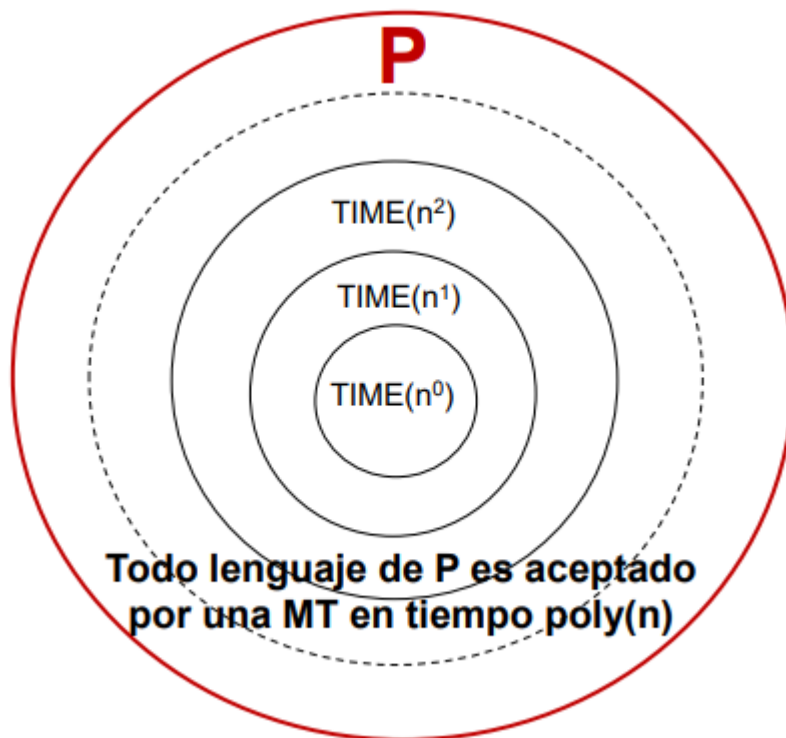


¿Es tratable n^{100} ?
 ¿Es intratable $1,001^n$?
 De todas maneras, estos valores no se dan en la práctica

Ejemplo, considerando una computadora determinada. Notar el salto abrupto en las funciones exponenciales:

	TAMAÑO					
	10	20	30	40	50	60
n	0,00001 segundos	0,00002 segundos	0,00003 segundos	0,00004 segundos	0,00005 segundos	0,00006 segundos
n ²	0,0001 segundos	0,0004 segundos	0,0009 segundos	0,0016 segundos	0,0025 segundos	0,0036 segundos
n ³	0,001 segundos	0,008 segundos	0,027 segundos	0,064 segundos	0,125 segundos	0,216 segundos
n ⁵	0,1 segundos	3,2 segundos	24,3 segundos	1,7 minutos	5,2 minutos	13,0 minutos
2 ⁿ	0,001 segundos	1,0 segundos	17,9 minutos	12,7 días	35,7 años	366 siglos
3 ⁿ	0,059 segundos	58,0 minutos	6,5 años	3855 siglos	2.10 ⁸ siglos	1,3.10 ¹³ siglos

Clase P



Robustez: si una MT M_1 con K_1 cintas tarda tiempo $\text{poly}(n)$, existe una MT M_2 equivalente con K_2 cintas que también tarda tiempo $\text{poly}(n)$

Ejemplo de lenguaje en P

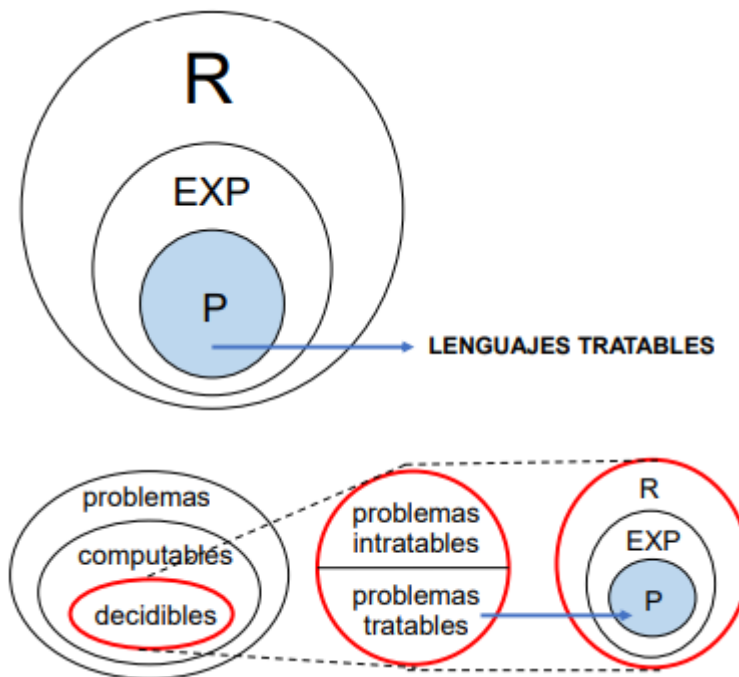
$L = \{w \mid w \text{ es un palíndromo (capicúa) con símbolos } a \text{ y } b\}$

Una MT muy simple que decide L , con 2 cintas, hace:

1. Copia w de la cinta 1 a la cinta 2 $\rightarrow$ Tiempo $O(n)$
2. Posiciona el cabezal de la cinta 1 a la izquierda $\rightarrow$ Tiempo $O(n)$
3. Compara en direcciones contrarias uno a uno los símbolos de las 2 cintas hasta llegar a un blanco en ámbas $\rightarrow$ Tiempo $O(n)$

Tiempo total: $T(n) = O(n) + O(n) + O(n) = O(n)$

Primera versión de la jerarquía temporal



R es la clase de los lenguajes recursivos

P es la clase de los lenguajes decidibles en tiempo $\text{poly}(n)$, es decir tiempo $O(n^k)$

EXP es la clase de los lenguajes decidibles en tiempo $\text{exp}(n)$, es decir tiempo $O(c^{\text{poly}(n)})$

Se prueba que $P \subset \text{EXP} \subset R$

Tesis Fuerte de Church-Turing (IMPORTANTE)

Si L es decidable en tiempo $\text{poly}(n)$ por un modelo computacional físicamente realizable, también es decidable en tiempo $\text{poly}(n)$ por una MT (al menos hasta que las máquinas cuánticas sean una realidad)

Ejemplo de lenguaje que no está en P

$\text{SAT} = \{\varphi \mid \varphi \text{ es una fórmula booleana sin cuantificadores satisfactible con } m \text{ variables}\}$

Nota: salvo mención explícita, trabajaremos con fórmulas booleanas sin cuantificadores

$\varphi_1 = (x_1 \wedge x_2 \wedge x_3) \vee (\neg x_1 \wedge \neg x_2 \wedge \neg x_3)$ es satisfactible con la asignación $\mathcal{A} = (V, V, V)$

$\varphi_2 = (x_1 \wedge x_2 \wedge x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3)$ no es satisfactible con ninguna asignación

Una MT M que decide SAT emplea una tabla de verdad (no se conoce otro algoritmo). P.ej., para φ_2 :

$(x_1 \wedge x_2 \wedge x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3)$						
V	V	V	F	V	V	V
V	V	F	F	V	V	F
V	F	V	F	V	F	V
V	F	F	F	V	F	F
F	V	V	F	F	V	V
F	V	F	F	F	V	F
F	F	V	F	F	F	V
F	F	F	F	F	F	F

2^m posibles asignaciones (en este caso 8)
 (por cada variable se prueba con los valores V y F)

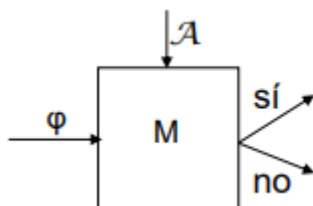
La evaluación de cada asignación
 se puede efectuar en tiempo $O(|\varphi_2|^2)$

Por lo tanto, M puede llegar a ejecutar $O(2^m \cdot |\varphi_2|^2) = O(2^n \cdot n^2) = \exp(n)$ pasos

Sin embargo...

$\text{SAT} = \{\varphi \mid \varphi \text{ es una fórmula booleana sin cuantificadores satisfactible con } m \text{ variables}\}$

Dada una fórmula booleana φ , una MT M puede llegar a decidir si $\varphi \in \text{SAT}$ en tiempo exponencial, pero contando con una asignación A , M puede verificar si $\varphi \in \text{SAT}$ en tiempo polinomial

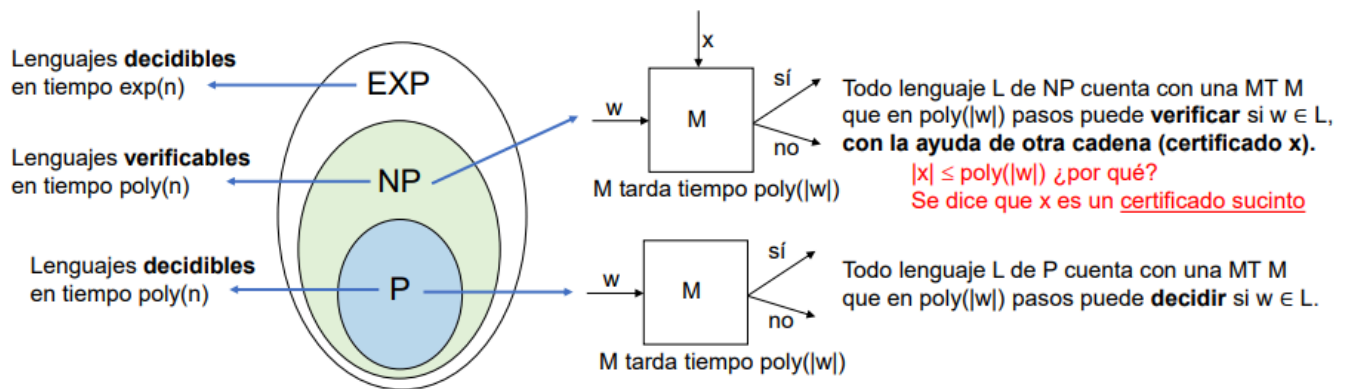


A es un **certificado** o **prueba** de φ .

M es un **verificador eficiente** del lenguaje SAT.

En tiempo $\text{poly}(|\varphi|)$, M puede verificar si $\varphi \in \text{SAT}$ con la ayuda de A

Segunda versión de la jerarquía temporal (acotada a la clase EXP)



En otras palabras:

- Si un problema está en P, podemos asegurar que sus soluciones se encuentran eficientemente
- Si un problema está en NP, sólo podemos asegurar que sus soluciones se verifican eficientemente

Intuitivamente, $P \neq NP$ (encontrar una solución parece ser más difícil que verificarla)

Sin embargo, al día de hoy esta relación no ha podido ser probada

P vs NP

El problema P vs NP fue planteado hace más de 50 años

Es uno de los siete problemas del milenio (definidos en el año 2000)

Quien lo resuelva recibirá un premio de USD 1 MM del Instituto Clay (EEUU)

No parece que pueda resolverse en el corto plazo, con las matemáticas existentes

Miles de lenguajes de interés están hoy día en NP – P (lenguajes relacionados con problemas de la lógica, grafos, aritmética, teoría de conjuntos, álgebra, combinatoria, redes, etc)

Pareciera que la única manera de decidir los lenguajes hoy día en NP – P es por medio de la fuerza bruta (búsqueda exhaustiva en el espacio de todas las soluciones posibles)

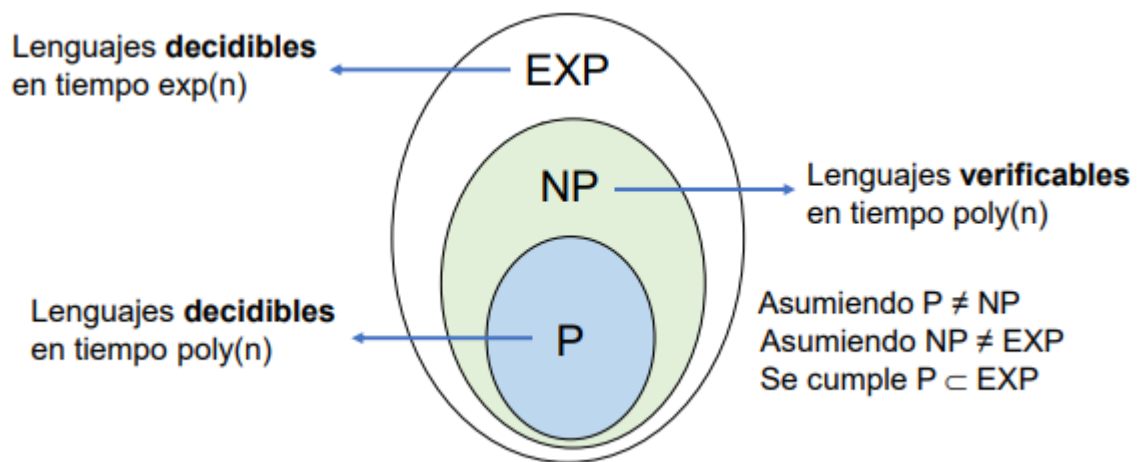
Por el contrario, los algoritmos que deciden los lenguajes de P son analíticos (parten del conocimiento profundo del problema, y se han elaborado con el empleo de ingenio, creatividad, experiencia, etc)

La mayoría de los investigadores opina que $P \neq NP$. Una minoría opina lo contrario, con el argumento de que queda mucho por aprender de la algorítmica (tiene asidero, ya que a lo largo de los años han habido importantes descubrimientos - primalidad en P , mejora en los tiempos del producto de matrices, camino en grafos no dirigidos en $LOGSPACE$, etc.)

En la práctica se asume $P \neq NP$, y por eso a los lenguajes de interés hoy día en $NP - P$ se los procesa con remediaciones específicas (cadenas de cierta longitud y/o forma, aproximaciones polinomiales, etc)

Clase 6 - NP-completitud

Jerarquía temporal hasta ahora

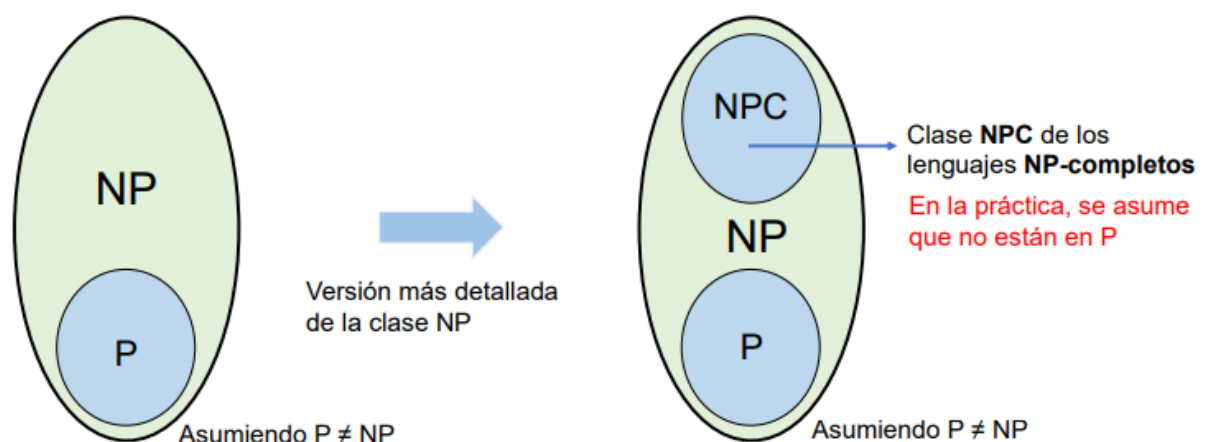


L está en P si y sólo si existe una MT M que para todo w : $w \in L$ si y sólo si M acepta w en tiempo $\text{poly}(n)$

L está en NP si y sólo si existe una MT M que para todo w : $w \in L$ si y sólo si existe x tal que M acepta (w, x) en tiempo $\text{poly}(n)$

Lenguajes NP-completos

Una manera de reforzar la sospecha de que un lenguaje de NP no está en P es probando que es NP-completo

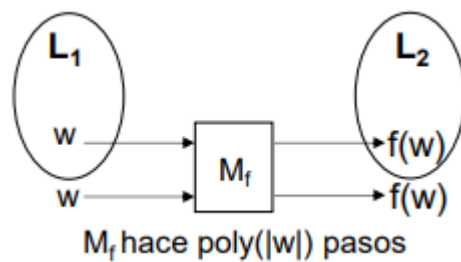


Se prueba que si un lenguaje NP-completo está en P, se cumple $P = NP$

Por lo tanto, si se cumple $P \neq NP$, un lenguaje NP-completo no está en P

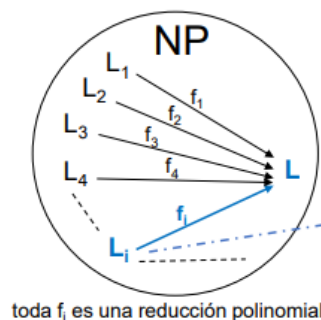
Definición de NP-completitud

Una reducción polinomial de un lenguaje L_1 a un lenguaje L_2 es una reducción de L_1 a L_2 de tiempo polinomial ($L_1 \leq_p L_2$)

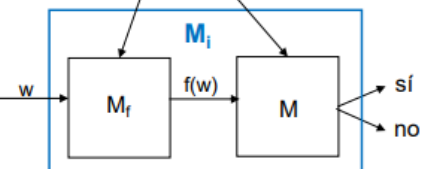


Un lenguaje L es NP-completo, o $L \in NPC$, si y sólo si:

1. $L \in NP$
2. Todo $L_i \in NP$ cumple $L_i \leq_p L$ (se dice que L es NP-difícil)



Teorema: si $L_i \leq_p L$ y $L \in P$, entonces $L_i \in P$

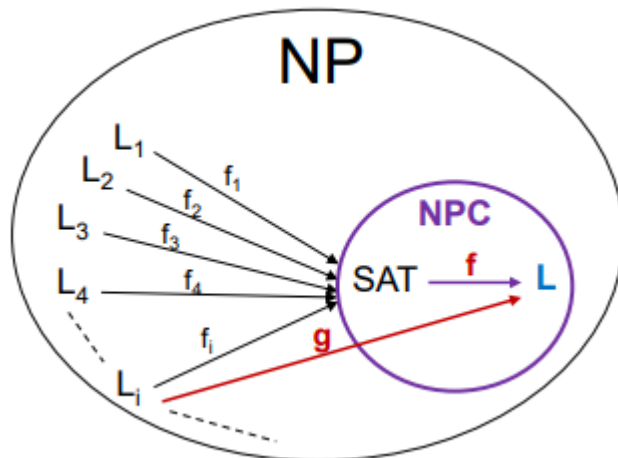


M_f hace $\text{poly}(|w|)$ pasos y M hace $\text{poly}(|f(w)|)$ pasos, con $|f(w)| \leq \text{poly}(|w|)$
 Por lo tanto, M_i decide L_i en $\text{poly}(|w|) + \text{poly}(\text{poly}(|w|)) = \text{poly}(|w|)$ pasos

Si L está en P, todo L_i de NP está en P, es decir: $NP = P$

Así, si $P \neq NP$, entonces L no está en P

¿Cómo encontrar un lenguaje L que sea NP-completo?



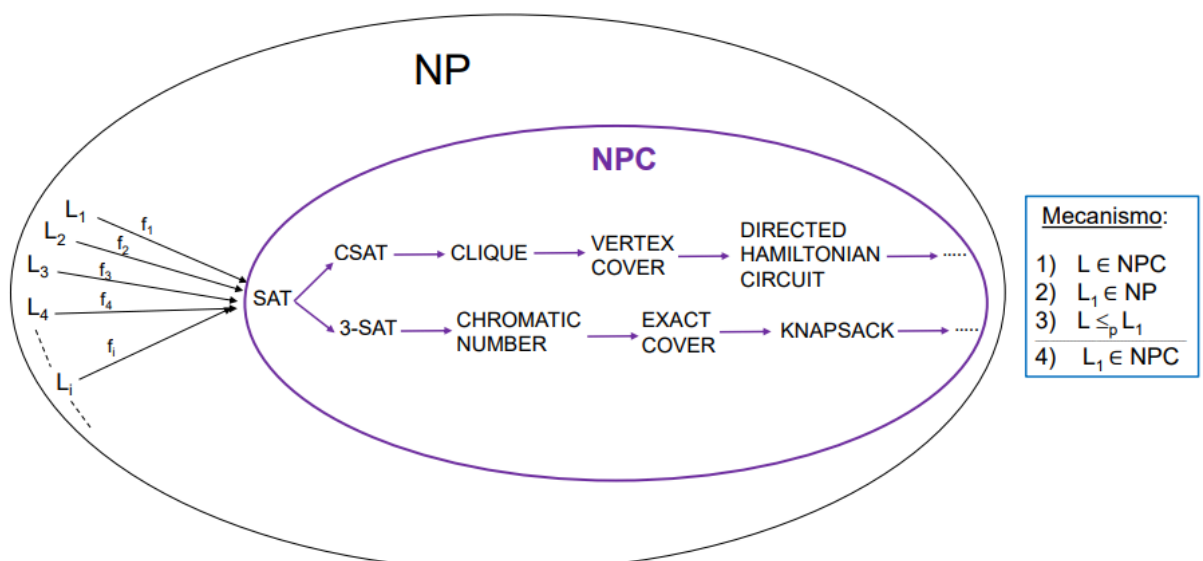
1. Probar que L pertenece a NP
2. Construir una reducción polinomial f de SAT a L

Se cumple que L es NP-completo:

- Sea g la composición de f_i con f
- La función g es una reducción polinomial de L_i a L (las reducciones polinomiales son transitivas)
- Como lo anterior vale para todo L_i de NP, entonces L es NP-completo

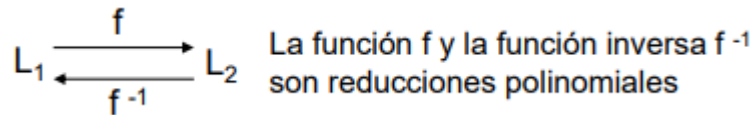
Repaso del mecanismo:

1) $SAT \in NPC$ 2) $L \in NP$ 3) $SAT \leq_p L$ $\rightarrow$ 4) $L \in NPC$



Observaciones

1. Todo par de lenguajes L_1 y L_2 NP-completos conocidos son p-isomorfos:



Se prueba que si todos los lenguajes NP-completos son p-isomorfos, se cumple $P \neq NP$

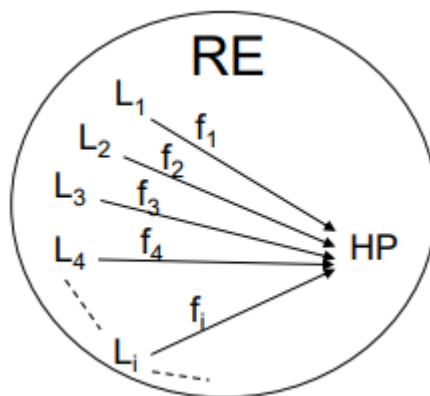
2. Todos los lenguajes NP-completos conocidos son densos, es decir que tienen muchas cadenas (formalmente, para todo n tienen $\exp(n)$ cadenas de longitud a lo sumo n)

A diferencia de lo indicado en la parte de computabilidad, en la complejidad computacional el tamaño de un lenguaje está muy ligado a su dificultad

Se prueba que si existe un lenguaje NP-completo no denso (disperso), se cumple $P = NP$

Los lenguajes completos de una clase (RE, NP u otra) identifican la dificultad de toda la clase. P.ej.:

- En la computabilidad:

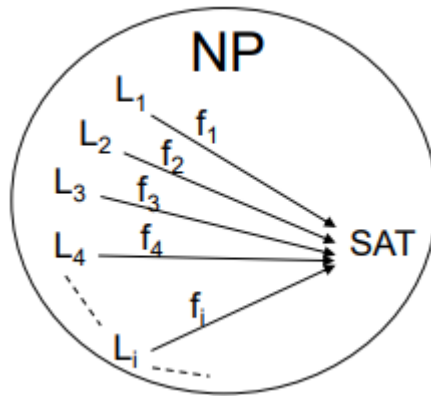


Todo lenguaje de RE se reduce a HP

HP es tan o más difícil que todos los lenguajes de RE (si HP fuera recursivo, entonces todos los lenguajes de RE serían recursivos, y así valdría $R = RE$)

HP (entre otros) identifica la dificultad de la clase RE

- En la complejidad temporal:



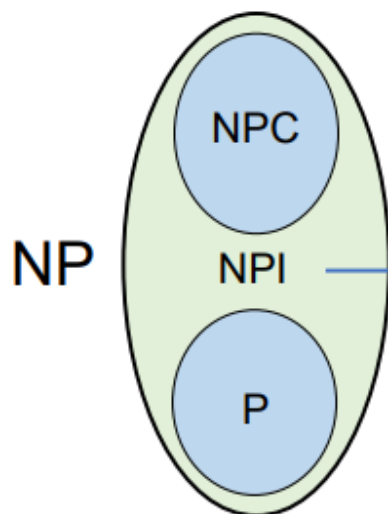
Todo lenguaje de NP se reduce polinomialmente a SAT

SAT es tan o más difícil que todos los lenguajes de NP (si SAT estuviera en P, entonces todos los lenguajes de NP estarían en P, y así valdría $P = NP$)

SAT (entre otros) identifica la dificultad de la clase NP

La clase NPI

Asumiendo $P \neq NP$, entre las clases P y NP^C de la clase NP se encuentra la clase NPI, llamada así porque identifica a los lenguajes de NP de dificultad intermedia (Teorema de Ladner, 1975)

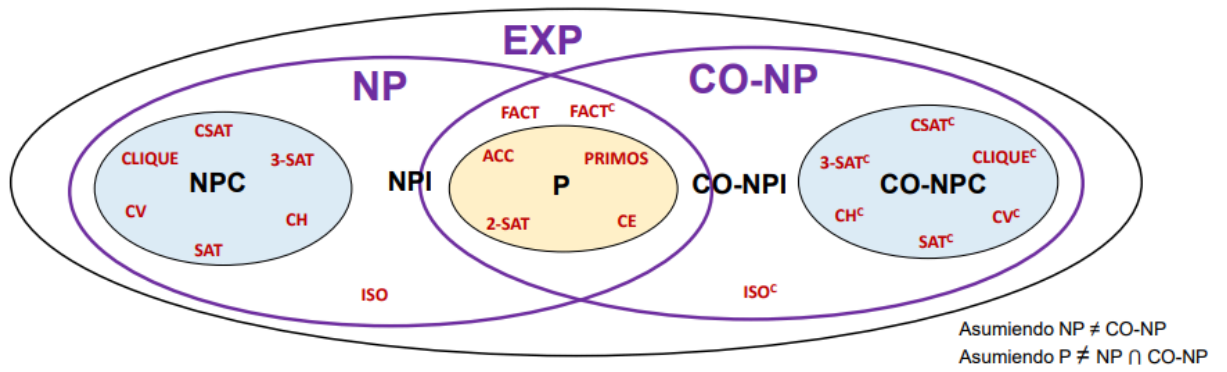


Asumiendo $P \neq NP$

Hoy día, en NPI se ubican los lenguajes que:

- No cuentan con una MT de tiempo polinomial que los decida.
- No cuentan con una reducción polinomial desde un lenguaje NP-completo.

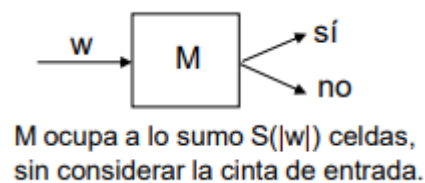
Última versión de la jerarquía temporal



Clase 7 - La jerarquía espacio temporal y otros temas de complejidad computacional

Complejidad espacial

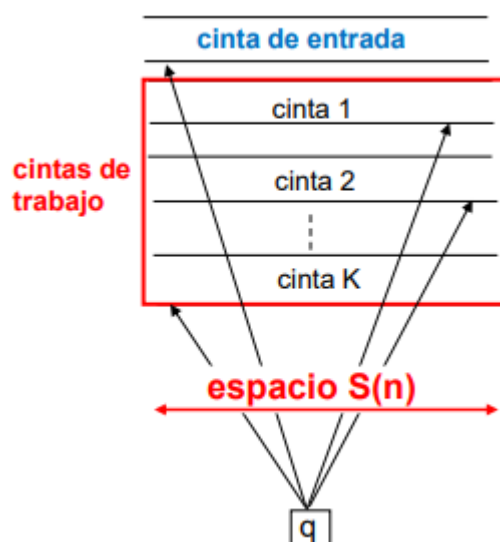
Una MT M ocupa espacio $S(n)$ si y sólo si al ejecutarse desde toda entrada w , con $|w| = n$, M ocupa a lo sumo $S(n)$ celdas en cualquier cinta distinta de la de entrada



La cinta de entrada es de sólo lectura y no se considera en la medición del espacio

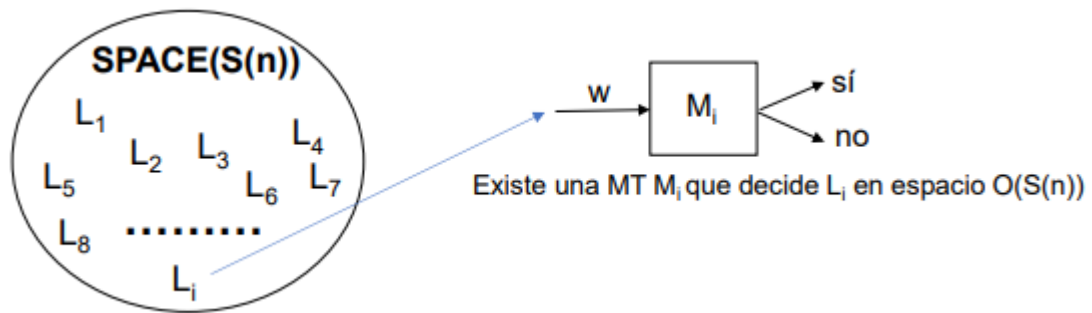
Esto permite un espacio menor que lineal, es decir menor que $O(n)$ (la cadena de entrada mide n)

Dada una MT M_1 de espacio $S(n)$ con varias cintas de trabajo, existe una MT M_2 equivalente de espacio $S(n)$ con una sola cinta de trabajo



En el caso más general con una cinta de salida, ésta tampoco se considera en la medición del espacio

Un lenguaje pertenece a la clase $SPACE(S(n))$ si y sólo si existe una MT M que lo decide en espacio $O(S(n))$:



Notar que:

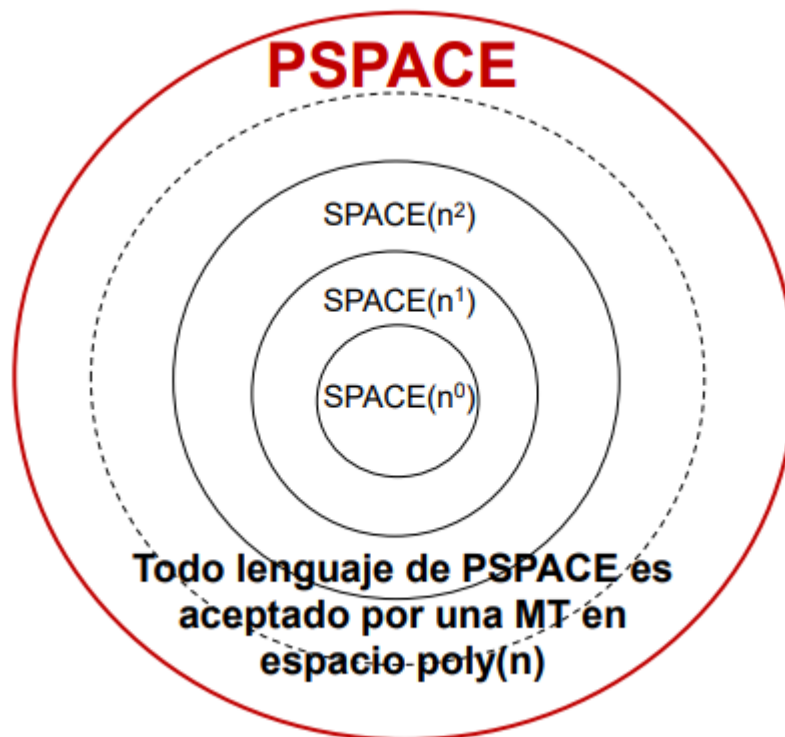
1. Una MT M de tiempo $T(n)$ ocupa a lo sumo espacio $T(n)$ celdas (porque al hacer $T(n)$ pasos, solo podrá ocupar como máximo $T(n)$ celdas, si se moviera siempre en la misma dirección)
2. En cambio, una MT M que ocupa espacio $S(n)$ puede tardar mucho más que tiempo $S(n)$:SA

Por ejemplo, si M tiene:

- Una cinta de entrada de solo lectura
- Una cinta de trabajo
- $|Q|$ estados
- $|\Gamma|$ símbolos

M puede llegar a hacer, si no loopea: $(n+2)*S(n)*|Q|*|\Gamma|^{S(n)} \leq c^{S(n)} = \exp(S(n))$ pasos

La clase PSPACE



Robustez: si una MT M_1 con K_1 cintas ocupa espacio $\text{poly}(n)$, existe una MT M_2 equivalente con K_2 cintas que también ocupa espacio $\text{poly}(n)$

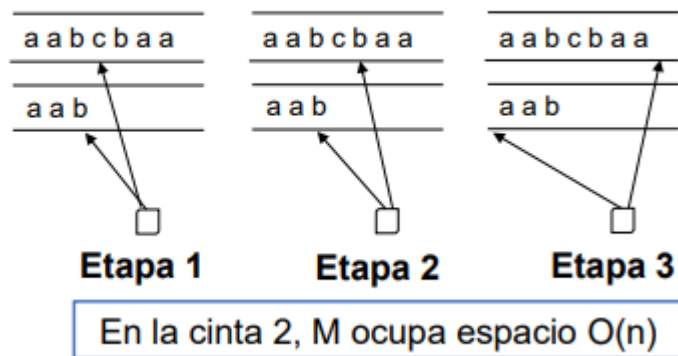
Ejemplo de lenguaje en la clase PSPACE

$L = \{vcv^R \mid v^R \text{ es la inversa de } v \text{ y tiene símbolos } a \text{ y } b\}$

La siguiente MT M decide L en espacio $O(n)$:

1. Copia la entrada hasta la c exclusive en la cinta 2
2. Saltea la c en la cinta 1
3. Compara símbolo a símbolo las dos cintas, yendo a la derecha en la cinta 1 y a la izquierda en la cinta 2. hasta llegar a un blanco en las dos cintas, en cuyo caso acepta

Por ejemplo:



La clase LOGSPACE

En realidad, $L = \{v \mid v \in R \mid v \text{ es la inversa de } v \text{ y tiene s\u00edmbolos } a \text{ y } b\}$ se puede decidir en menos espacio

La siguiente MT decide L en espacio $O(\log_2 n)$

Ve\u00e1moslo por ejemplo con la entrada $w = aabbbcbbaa$:

1. Hace $i := 1$ en la cinta 1
2. Hace $j := n$ en la cinta 2. Si j es par, rechaza
3. Copia el s\u00edmbolo i de w en la cinta 3
4. Copia el s\u00edmbolo j de w en la cinta 4
5.
 - a. Si $i = j$: si los s\u00edmbolos son c , acepta, si no, rechaza
 - b. Si $i \neq j$: si los s\u00edmbolos no son los dos a o los dos b , rechaza
6. Hace $i := i + 1$ en la cinta 1
7. Hace $j := j - 1$ en la cinta 2
8. Vuelve al paso 3

La MT M ocupa el espacio de los contadores i y j , que en binario miden $O(\log_2 n)$, m\u00e1s 2 celdas para alojar cada vez a los s\u00edmbolos comparados (espacio constante)

Por lo tanto, $L \in \text{SPACE}(\log_2 n)$

A esta clase se la denomina LOGSPACE

1ra iteración		2da iteración	
	a a b b b c b b b a a		a a b b b c b b b a a
1	i = 1	1	i = 2
2	j = 11	2	j = 10
3	a	3	a
4	a	4	a

.....

anteúltima iteración		última iteración	
	a a b b b c b b b a a		a a b b b c b b b a a
1	i = 5	1	i = 6
2	j = 7	2	j = 6
3	b	3	c
4	b	4	c

La jerarquía espacial

- LOGSPACE es la clase de los lenguajes decidibles en espacio $O(\log_2 n)$
- PSPACE es la clase de los lenguajes decidibles en espacio $\text{poly}(n)$
- EXPSPACE es la clase de los lenguajes decidibles en espacio $\exp(n)$

De antes: espacio $S(n)$ implica tiempo $c^{S(n)}$, con c constante

En particular: espacio $\log_2 n$ implica tiempo $c^{\log_2 n}$, con c constante

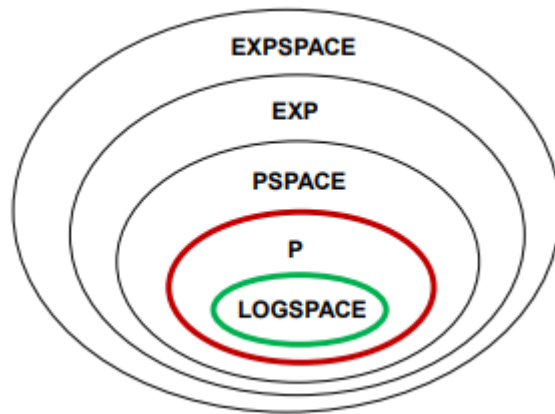
Pero: $c^{\log_2 n} = n^{\log_2 c}$, con c constante, es decir $\text{poly}(n)$

Por lo tanto: espacio $\log_2 n$ implica tiempo $\text{poly}(n)$

En definitiva, $\text{LOGSPACE} \subseteq P$, es decir que los lenguajes de LOGSPACE son tratables

Se prueba que $\text{LOGSPACE} \subset \text{PSPACE}$ Se conjetura que $\text{LOGSPACE} \subset P$

Se conjetura que $P \subset \text{PSPACE}$



La jerarquía espacio-temporal

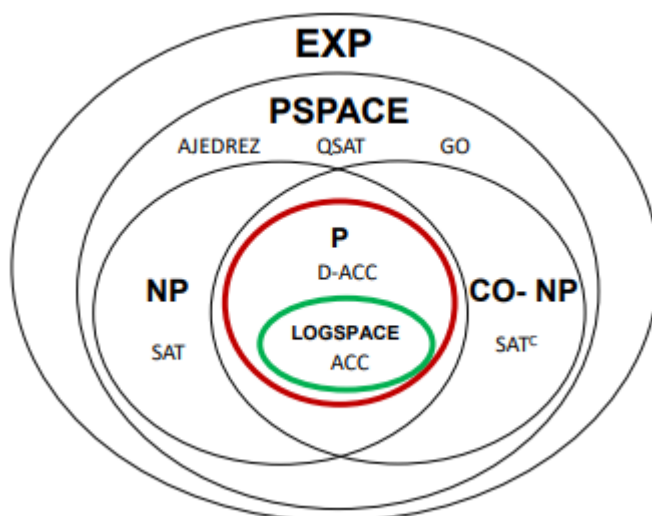
QSAT está entre los lenguajes más difíciles de PSPACE

Es PSPACE-completo (todo $L \in \text{PSPACE}$ cumple $L \leq_p \text{QSAT}$)

Otros problemas PSPACE-completos clásicos se asocian a típicos juegos entre dos jugadores J_1 y J_2 , que plantean decidir si J_1 gana:

- ¿Existe una jugada de J_1 tal que para toda jugada de J_2 existe una jugada de J_1 tal que para toda jugada de J_2 ... J_1 gana?
- Ejemplos: el ajedrez y el go (asumiendo tableros de $n \times n$ entre otras cosas)

Por otro lado, $\text{ACC} = \{G \mid G \text{ es un grafo no dirigido con un camino del vértice 1 al vértice } m\}$ está en LOGSPACE, y pareciera que el mismo lenguaje pero definido con grafos dirigidos es más difícil: $\text{D-ACC} = \{G \mid G \text{ es un grafo dirigido con un camino del vértice 1 al vértice } m\}$ no estaría en LOGSPACE



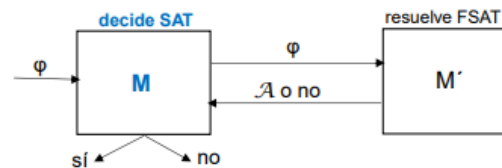
Complejidad temporal de búsqueda

Las clases P y NP correspondientes a los problemas de búsqueda (o problemas de función) se denominan FP y FNP

Para distinguirlos, a los nombres de los problemas de búsqueda se les antepone una F

Ejemplo. Problemas FSAT (búsqueda) y SAT (decisión)

Claramente, FSAT es tan o más difícil que SAT:



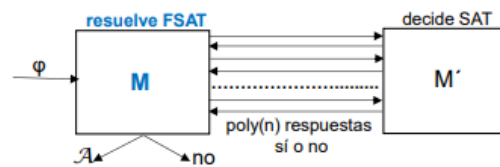
A partir de M' se puede construir M , tal que:
si M' resuelve FSAT en tiempo $\text{poly}(n)$,
entonces M decide SAT en tiempo $\text{poly}(n)$.

Formalizando:

Si $\text{FSAT} \in \text{FP}$ entonces $\text{SAT} \in \text{P}$. O lo mismo:

Si $\text{SAT} \notin \text{P}$ entonces $\text{FSAT} \notin \text{FP}$.

Menos intuitivo, también se cumple que SAT es tan o más difícil que FSAT:



A partir de M' se puede construir M , tal que:
si M' decide SAT en tiempo $\text{poly}(n)$,
entonces M resuelve FSAT en tiempo $\text{poly}(n)$.

Formalizando:

Si $\text{SAT} \in \text{P}$ entonces $\text{FSAT} \in \text{FP}$. O lo mismo:

Si $\text{FSAT} \notin \text{FP}$ entonces $\text{SAT} \notin \text{P}$.

Esto se cumple **para todos los lenguajes NP-completos**. También se cumple **$\text{P} = \text{NP}$ sii $\text{FP} = \text{FNP}$** .

Aproximaciones polinomiales

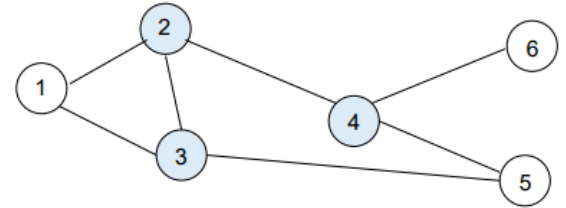
Cuando a un problema de búsqueda de óptimo (máximo o mínimo) no se le conoce resolución polinomial, se plantea resolverlo con una aproximación polinomial (MT de tiempo polinomial con una solución “buena”)

Ejemplo. Mínimo cubrimiento de vértices de un grafo (no tendría resolución polinomial).

La siguiente MT M, en el peor caso, encuentra un cubrimiento con el **doble** de vértices del mínimo:

Dado $G = (V, E)$, la MT M hace:

1. $V' := \emptyset$ y $E' := E$
2. Si $E' = \emptyset$, acepta
3. $E' := E' - \{(v_1, v_2)\}$, siendo (v_1, v_2) algún arco de E'
4. Si $v_1 \notin V'$ y $v_2 \notin V'$, entonces $V' := V' \cup \{v_1, v_2\}$
5. Vuelve al paso 2



Es decir, se toman arcos no adyacentes de E y se construye un cubrimiento de vértices con sus dos extremos.

El cubrimiento mínimo del grafo de arriba tiene **tamaño 3**. La MT M puede generar, entre otros cubrimientos: $V' = \{1, 2, 3, 5, 4, 6\}$, de **tamaño 6**, y $V' = \{1, 2, 4, 5\}$, de **tamaño 4** (ejercicio: ¿cuáles otros?).

Hay problemas con aproximaciones polinomiales muy buenas (la diferencia ϵ con el óptimo tiende a 0), y otros problemas que no cuentan con aproximaciones polinomiales (cualquier ϵ que se fije es superado).

Lenguajes con algoritmos paralelos eficientes

Modelo más usual de ejecución paralela: circuito booleano

Un circuito booleano acepta una cadena si y sólo si su salida es 1 (ver ejemplo)

Para manejar todos los tamaños de cadenas, se usan familias de circuitos

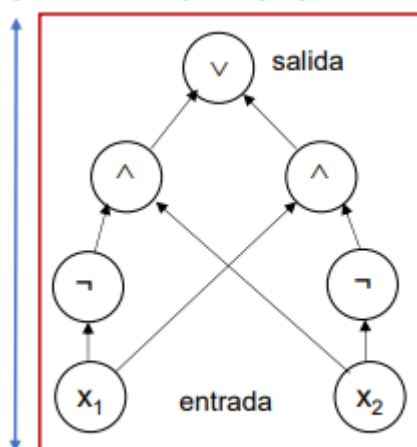
Se define que un lenguaje cuenta con un algoritmo paralelo eficiente, si y sólo si lo acepta una familia de circuitos de tamaño $\text{poly}(n)$ y profundidad $\log_k(n)$

La clase de estos lenguajes se llama NC (Nick class, por Nicholas Pippenger)

Problemas clásicos de NC: operaciones aritméticas, producto de matrices, búsqueda del camino mínimo en un grafo, ordenamiento de un vector, etc.

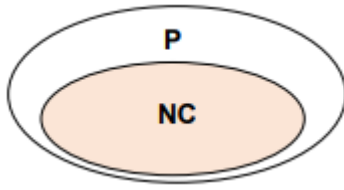
Se cumple $NC \subseteq P$. La conjetura más aceptada es que $NC \neq P$

profundidad ("tiempo paralelo")



tamaño (cantidad de puertas)

Evaluación de $(x_1 \wedge \neg x_2) \vee (x_2 \wedge \neg x_1)$



Problema clásico que estaría en $P - NC$: máximo flujo en una red

MT probabilísticas o MTP (algoritmos aleatorios)

Una MT probabilística (MTP), en cada paso elige aleatoriamente una entre dos continuaciones, cada una con probabilidad $1/2$ (“tiro de moneda”)

Un lenguaje L pertenece a la clase BPP (bounded probabilistic polynomial) si y sólo si existe una MTP M con computaciones de tiempo $\text{poly}(n)$ tal que:

- Si $w \in L$, M acepta w en al menos $\frac{2}{3}$ de sus computaciones
- Si $w \notin L$, M rechaza w en al menos $\frac{2}{3}$ de sus computaciones

Por lo tanto, las MTP asociadas a BPP tienen una probabilidad de error $\epsilon \leq \frac{1}{3}$

A la clase BPP se la considera la clase P probabilística

Existen problemas que hoy día no cuentan con MT eficientes, pero sí MTP eficientes

Por ejemplo, el problema de decidir si dos polinomios multivariados son iguales

Que PRIMOS esté en P , no significa que no se use la MTP descrita recién (es más rápida y simple)

$P \subseteq BPP$ (una MT es un caso particular de MTP, que en todo par de opciones hace lo mismo)

La conjetura más aceptada es que $P = BPP$

Es decir, todo algoritmo probabilístico polinomial se podría desaleatorizar con un retardo sólo polinomial, por la posibilidad de simular lo aleatorio con generadores pseudoaleatorios, que generan a partir de pequeñas cadenas aleatorias (semillas) grandes cadenas que “parecen” aleatorias

Máquinas cuánticas o MC (algoritmos cuánticos)

Las máquinas cuánticas (MC), a diferencia de las máquinas clásicas, tienen qubits en lugar de bits

Un cúbit, como un bit, puede estar en los estados básicos 0 o 1, pero a diferencia de un bit, puede estar también en un estado de superposición de 0 y 1. Los estados de un cúbit se suelen anotar así:

<u>Estados posibles de un cúbit</u>		
$ 0\rangle$	$ 1\rangle$	$\alpha_0 0\rangle + \alpha_1 1\rangle$

En el último caso, α_0 y α_1 son números complejos (amplitudes) que cumplen $|\alpha_0|^2 + |\alpha_1|^2 = 1$, tal que:

- El cúbit está en un estado de superposición de 0 y 1
- Al medirlo (leerlo), se obtiene el valor 0 con probabilidad $|\alpha_0|^2$ o el valor 1 con probabilidad $|\alpha_1|^2$
- Luego de la medición se destruye la superposición (el cúbit queda en el estado básico obtenido, 0 o 1)

Así, utilizando registros con varios cubits, la capacidad computacional de las MC se torna muy grande:

- 1 cúbit guarda la misma información que 2 bits: $\alpha_0|0\rangle + \alpha_1|1\rangle$
- 2 cubits lo mismo que 4 pares de bits: $\alpha_0|00\rangle + \alpha_1|01\rangle + \alpha_2|10\rangle + \alpha_3|11\rangle$
- 3 cubits lo mismo que 8 ternas de bits: $\alpha_0|000\rangle + \alpha_1|001\rangle + \dots + \alpha_7|111\rangle$
- Etc.

En el caso de una MC con un registro de 3 cubits, por ejemplo: la MC contempla 8 estados superpuestos hasta medir el registro, luego de lo cual se queda con un estado si con probabilidad $|\alpha_i|^2$, perdiéndose la superposición

Los cambios en los cúbits se hacen con operaciones o puertas cuánticas

Una puerta cuántica se aplica sobre uno, dos o tres cúbits

La cantidad de puertas ejecutadas establece el tiempo de la MC

La clase BQP (bounded-error quantum polynomial time), contiene los lenguajes aceptados por MC de tiempo $\text{poly}(n)$, con probabilidad de error $\leq \frac{1}{3}$. Como las MTP, las MC pueden equivocarse, pero la probabilidad de error, repitiendo ejecuciones, es muy baja

BQP es la clase P cuántica

Se cumple $P \subseteq BQP$ y también $BPP \subseteq BQP$. La conjetura más aceptada es que las dos inclusiones son estrictas

Por lo tanto, las MC podrían refutar la Tesis Fuerte de Church-Turing. En el caso de que hubiera algún lenguaje L , decidible en tiempo $\text{poly}(n)$ por un modelo computacional físicamente realizable (cuántico), pero no decidible en tiempo $\text{poly}(n)$ por una MT

Otra conjetura aceptada es que BQP y NP son incomparables (ninguna clase incluye a la otra)

También se cumple $BQP \subseteq PSPACE$

Un ejemplo de problema en BQP que hoy día no tiene un algoritmo clásico polinomial es la factorización. El algoritmo cuántico de P. Shor resuelve la factorización en tiempo polinomial

Las MC no podrían decidir eficientemente los lenguajes NP -completos. El algoritmo de L. Grover acelera “sólo” cuadráticamente el mejor algoritmo clásico para resolver SAT

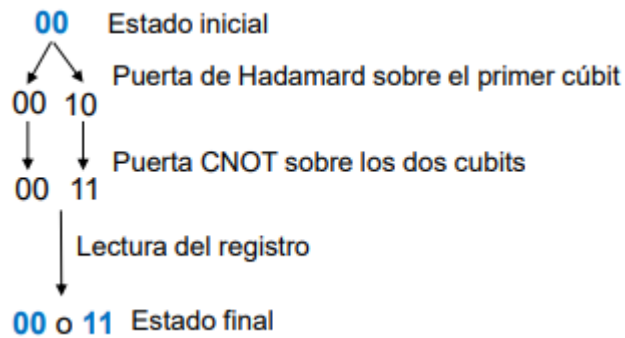
Las MC en principio son factibles físicamente. Un aspecto a considerar muy relevante es el “ruido” (interferencias), que atenta contra las superposiciones de estados. Para combatirlo se vienen desarrollando distintos procedimientos de manejo de errores

Ejemplo de puertas cuánticas y computación cuántica

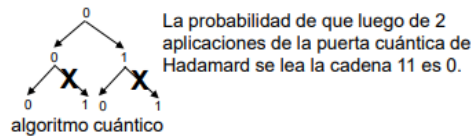
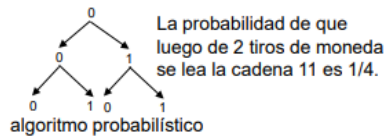
Puerta de Hadamard: dado un cúbit en estado básico 0 o 1, lo pasa al estado de superposición 0 y 1

Puerta CNOT: dados dos cubits, invierte el estado del 2do sólo si el 1ro es 1

1. Sea un registro cuántico de dos cubits en el estado $|00\rangle$
2. Aplicando sobre el primer cúbit la puerta de Hadamard, se obtiene la superposición de los estados $|00\rangle$ y $|10\rangle$
3. Aplicando sobre los dos cubits la puerta CNOT, se obtiene la superposición de los estados $|00\rangle$ y $|11\rangle$
4. Finalmente, leyendo el registro, se obtiene el estado $|00\rangle$ o $|11\rangle$, cada uno con probabilidad $\frac{1}{2}$, y se destruye la superposición



Diferencia entre los algoritmos probabilísticos y los algoritmos cuánticos



Sólo en las MC las computaciones **se interfieren**. Esto marcaría su **real ventaja** sobre las MTP.

Las distintas jerarquías

